

Specifying, Tiling, and Fusing Complex Reductions

by

Steven N. Raphael

Submitted to the Department of Electrical Engineering and Computer Science
in partial fulfillment of the requirements for the degree of

MASTER OF ENGINEERING IN ELECTRICAL ENGINEERING AND
COMPUTER SCIENCE

at the

MASSACHUSETTS INSTITUTE OF TECHNOLOGY

May 2026

© 2026 Steven N. Raphael. All rights reserved.

The author hereby grants to MIT a nonexclusive, worldwide, irrevocable, royalty-free license to exercise any and all rights under copyright, including to reproduce, preserve, distribute and publicly display copies of the thesis, or release the thesis under an open-access license.

Authored by:	Steven N. Raphael Department of Electrical Engineering and Computer Science May 15, 2026
Certified by:	Saman Amarasinghe Department of Electrical Engineering and Computer Science, Thesis Supervisor
Accepted by:	Katrina LaCurts Chair Master of Engineering Thesis Committee

Specifying, Tiling, and Fusing Complex Reductions

by

Steven N. Raphael

Submitted to the Department of Electrical Engineering and Computer Science
on May 15, 2026 in partial fulfillment of the requirements for the degree of

MASTER OF ENGINEERING IN ELECTRICAL ENGINEERING AND
COMPUTER SCIENCE

ABSTRACT

The Halide programming language is a domain-specific language that separates a function’s definition from its schedule. However, more complex optimization patterns involving tiling and recursive functions are difficult or even impossible to express in Halide. To meet the challenge of expressing complex optimization strategies, a scheduling language, together with the underlying language it augments, should have both the representational power to express the underlying dependency patterns of the optimized schedule and the power of abstraction to write out high-level optimizations in a natural form. We present two novel mechanisms to focus on specific aspects of both requirements: one feature to improve the abstract representation of tiling, and another to express a common pattern involving loop fusion of recursive functions. Although Halide has scheduling techniques that perform tiling in simple cases, more complex tiling transformations need to be written into the function definition rather than the schedule, which is inelegant and prone to bugs. We design and implement *FlashTile* in Halide, backed by a theoretical framework, to semi-automatically perform complex tiling transformations, applying the system to both reductions and sliding-window aggregations. We also describe modifications to the Halide compiler that were necessary for FlashTile to produce efficient code. Our experiments show that FlashTile can replicate hand-optimized schedules such as box filter pipelines. Implementing a novel optimization, FlashTile produces up to an 8x speedup over PyTorch for a transformer-based time-series model. Next, we design and implement **inductive functions** in Halide. This extension allows Halide to express loop fusion over recursively defined functions where the loop index is one of the recursive variables. We present several algorithms where inductive functions allow Halide to express more performant schedules that are otherwise impossible to implement.

Thesis supervisor: Saman Amarasinghe

Title: Department of Electrical Engineering and Computer Science

Acknowledgments

I would like to thank my graduate student supervisor Teo, who provided tremendous help guiding me on the right path and turned my originally vague ideas into a coherent plan. I would also like to thank Saman for his very useful feedback, and my family for all their support.

Contents

<i>List of Figures</i>	9
<i>List of Tables</i>	11
1 Introduction	13
1.1 Outline	14
2 Background and related work	17
2.1 Halide and similar DSLs	17
2.2 Tiling	18
2.3 Attention and FlashAttention	19
2.4 Sliding windows	20
2.5 Loop fusion	20
2.5.1 Loop fusion in Halide	21
3 FlashTile theory	25
3.1 Simple examples	25
3.2 Description of transformations	29
3.3 Algorithms for processing tiles	31
3.4 Derivation of merge functions	33
3.5 Merge function existence	33
3.6 Dot product example	33
4 FlashTile infrastructure	35
4.1 Scheduling preconditions and recognizing functions in Halide	35
4.2 Merge functions and their synthesis	36
4.3 Combining tiles	37
4.4 Modifications to Halide	40
4.5 Problems with FlashAttention	42
5 Inductive functions	43
5.1 Construction and implementation	43
5.1.1 Definition	43
5.1.2 Compilation	45
5.1.3 Fibonacci example	47

6	Evaluation	49
6.1	FlashTile for FlashAttention	49
6.1.1	Construction	51
6.1.2	Experimental results	53
6.2	FlashAttention in a time series	54
6.2.1	Construction	55
6.2.2	Experimental results	56
6.3	FlashTile in the StereoBM algorithm	56
6.3.1	Construction	58
6.3.2	Experimental results	58
6.4	Benchmarks	60
6.5	Inductive functions	60
7	Future Work	63
7.1	Future Work on FlashTile	63
7.2	Future work on inductive functions	63
7.2.1	Parallelization and vectorization	63
7.2.2	General recursion	63
7.2.3	Improved storage folding	64
A	Additional material	65
A.1	Proofs	65
A.2	Preparation of Call Tree Details	66
	<i>References</i>	69

List of Figures

2.1	Example application of <code>rfactor</code>	19
2.2	FlashAttention algorithm	20
2.3	Sliding window algorithm	21
2.4	Dependency graph of a two-stage pipeline	21
2.5	Comparison of a single-definition recursive function (top) and a more complicated update (bottom)	22
2.6	Simplified version of a poorly optimized box sum computation in large tiles. Step 1: Compute initial values at the edge of each tile. Step 2: Recursively compute 1-dimensional box sums by adding a new value and subtracting an old value at each step. Step 3: Recursively compute 2-dimensional box sums in the same manner.	22
2.7	Dependency graph and memory storage (in gray) of OpenCV's StereoBM implementation at each loop iteration	23
5.1	Halide's compilation pipeline. The boxes describe general steps in Halide's compilation process. The text below each box describes compilation steps that are specific to inductive functions.	43
5.2	Comparison of computed values for an inductively defined Fibonacci function with and without various compiler optimizations. The filled-in boxes represent computed values during each loop iteration.	48
6.1	Memory transfers for unoptimized attention and FlashAttention.	54
6.2	Runtimes of sliding attention in Halide and regular FlashAttention in PyTorch	57
6.3	StereoBM schedule results	59
6.4	Runtime comparison for three implementations of the StereoBM algorithm	60
6.5	Runtime comparison for two implementations of the two-stage pipeline in Figure 2.4.	62

List of Tables

2.1	Comparison of tensor DSLs and scheduling models	18
6.1	Using FlashTile reduces the lines of code in the benchmarks. This comparison counts lines containing scheduling commands as well as lines used to define functions.	60

Chapter 1

Introduction

A scheduling language expresses a *schedule* that describes what optimizations should be performed to a program at the compilation stage. Many scheduling languages have been created and published in the literature. To meet the challenge of expressing complex optimization strategies, a scheduling language should have both the representational power to express the underlying dependency patterns of the optimized schedule and the power of abstraction to write out high-level optimizations in a natural form.

One common transformation pattern used across many different scheduling languages is tiling. Tiling can improve locality and parallelism. Many tiled forms of algorithms are divided into an intermediate stage and a merge stage [1]: the intermediate stage independently computes an output for every tile, and the merge stage combines the outputs from every tile into a single output. Unfortunately, manually writing out tiled programs is a process that can be ugly and error-prone. It would be much simpler to write out an untiled version of a program and have an abstraction that semi-automatically transforms it into a tiled version. Program synthesis, as applied to automatic tiling, is an active area of research [2]. There is also a lot of research into creating compilers that perform tiling optimizations [3] [4].

Halide [5] is a domain-specific language that separates a function’s definition (what is to be computed) from its computation schedule (how the computation is performed). It is one of the first scheduling languages and a popular framework for optimizing complex pipelines. The functions in Halide operate mainly over dense arrays. These commands include basic ones such as the `compute_at` command which schedules a function to be computed at a certain position in the loop nest of another function. It also includes tiling commands for both simple commands and certain reductions. In particular, the `rfactor` command [1] is used to tile associative reductions: functions whose input is an array and whose output is a single value (or a constant-sized tuple of values).

Halide’s tiling methods are limited to two cases: either the computation to be tiled has no dependencies over the dimension that is tiled, or the computation takes the form of an associative reduction: a function whose input is an array and whose output is a single value (or a constant-sized tuple of values). Halide is currently unable to synthesize tiled forms of more complex pipelines; it requires the tiled forms to be written in the definition. It is also unable to create efficient tiled forms of algorithms that are not just simple reductions.

In this paper, we first created a system, *FlashTile*, that enables semi-automatic tiling of computations and recombination of tile outputs into a final result, preventing programmers

from having to manually write out the algorithm’s tiled form. FlashTile is integrated into the Halide scheduling language. It uses multiple algorithmic templates to tile different types of algorithms. FlashTile is based on theoretical description and analysis of the tiling process, which includes the process of generating merge functions. However, we realized that FlashTile was insufficient to schedule FlashAttention in Halide.

The Halide language, not just its scheduling capabilities, was one of the barriers that prevented FlashAttention from being represented in Halide. The problems could not be resolved by adding more abstractions. This is because FlashAttention relies on another very common scheduling optimization: loop fusion. This optimization combines computations across multiple loops into a single loop, which can reduce memory usage and improve locality. Although Halide’s `compute_at` scheduling command allows for loop fusion in many cases, it cannot perform loop fusion in cases where the functions are recursively defined and do recursion over the loop index. This is a necessary optimization in the FlashAttention algorithm, and this issue is discussed in greater detail in Section 4.5.

To remedy the problems with FlashTile, we decided to modify the Halide language by extending Halide’s idea of pure functions to implement common recursive patterns. Inductive functions can be used to represent optimal schedules for several different algorithms that are otherwise impossible to express in Halide. We show that our extension integrates well with Halide’s existing features and implements an important type of optimization that is not possible in many other scheduling languages. Our approach differs from FlashTile as well as most other work on scheduling languages, which has tended to develop ways to improve scheduling rather than improving the language itself.

1.1 Outline

In Section 3, we present the ideas behind FlashTile, demonstrating the theoretical methods we use to construct merge functions and algorithms for tiling reductions and sliding-window aggregations.

In Section 4, we describe how we implemented FlashTile in Halide. FlashTile consists of three parts: dividing the function computation into tiles, deriving a merge function that combines the results of two tiles, and replacing the original function’s definition with a function that combines the tiles using the merge function. This third step is where the implementation for tiling reductions differs from the implementation for tiling sliding-window aggregations. We also detail the modifications we made to the Halide compiler to enable FlashTile to be as efficient as possible.

In Section 6, we evaluate FlashTile in a range of domains, demonstrating not only that FlashTile can reproduce the schedules of manually optimized pipelines, but also that it streamlines the creation of new algorithms. We find that FlashTile can replicate the tiling strategy of FlashAttention from a simple but unoptimized definition of attention, and it can also replicate multiple optimized schedules for a box filter algorithm from a single definition. We also use FlashTile to implement a novel optimization for a transformer-based time series prediction model, demonstrating an 8x speedup over PyTorch.

In Section 5, we describe the design and implementation of our inductive functions feature. We demonstrate how inductive functions can be used with other Halide scheduling commands.

We walk through how inductive functions are compiled and how the Halide compiler’s existing optimizations work on inductive functions. We use inductive functions to create optimized schedules for two different algorithms that cannot otherwise be expressed in Halide.

In summary, this thesis makes the following contributions:

- We introduce a theoretical framework for implementing tiled algorithms. Among other things, this framework describes a way to generate a merge stage that combines the results of two tiles.
- We implement the framework in Halide to semiautomatically transform complex reductions and sliding-window aggregations from simple, unoptimized definitions into more efficient tiled forms.
- We use our infrastructure to optimize a transformer layer for time-series prediction.
- We show that FlashTile can implement tiling transformations in Halide with less code compared to manual implementations of the tiled functions.
- We define an extension of the Halide language to allow it to express a new kind of schedule.
- We show that our new extension can bring performance improvements to programs in Halide.

Chapter 2

Background and related work

In this section, we introduce preliminary concepts and describe algorithms that have already been developed.

2.1 Halide and similar DSLs

Halide [5] is a domain-specific language (DSL) that allows programmers to control the schedule of a computation after the functions involved in the computation have been defined. The programmer defines a function’s input and output in functional programming style without specifying which outputs need to be computed; these functions are defined for every possible input. The Halide compiler performs *bounds inference* to automatically determine the iteration space for each function. Functions can have *update stages*, where their outputs are modified after an initial *pure definition*. In update stages, functions can refer to themselves, which is necessary to write a lot of useful computations in Halide. Update stages must contain *reduction domains*, which control how the computation is ordered within an update definition. However, the focus of the Halide scheduling language is on pure functions with integer arguments.

Many other DSLs and similar tools exist for software optimization, which are summarized in Table 2.1. Most other scheduling languages do not have the same approach and generality as Halide. In particular, most languages lack the ability to schedule arbitrary pure functions that are defined everywhere. Some languages, such as RISE [6] and Exo [7], require the bounds of each function to be defined by the programmer. Fern [8] is a newer language designed for operator fusion and composition. It uses a bounds inference model similar to Halide but does not have any infrastructure for performing computations. Other scheduling languages operate on a more restrictive domain than Halide in order to solve a more specific problem. For example, TACO [9], ATL [10], and DISTAL [11] are designed to schedule programs that use tensor algebra, and they are limited to performing operations on einsums. Other languages, such as Tiramisu [12], focus on scheduling programs using the polyhedral model. The MDH scheduling language [13] comes the closest to Halide’s bounds inference idea, but it is focused on abstract data parallelism. MDH also has basic recursion in the form of a built-in prefix sum, but that is an ad hoc specialization that does not extend to general recursion. TVM [14] is another exception; this language extends the ideas of Halide to take

advantage of GPUs and hardware accelerators. A broader overview of scheduling languages is given in [15].

Paper	Language	Scheduling
Halide	Pure functions on infinite domains + reductions	Tiling, <code>compute_at</code> , <code>store_at</code> , CPU/GPU scheduling, sliding window
Lift/Rise	Pure functional language on finite arrays	Rewrite rules
TVM	Same as Halide	Adds tensor cores to Halide-style scheduling
Tiramisu	Polyhedral loop nests	Polyhedral scheduling commands
TACO	Einstein index notation	Subset of Halide + sparse-specific operations
DISTAL	Einstein index notation	Halide subset + distributed execution + skewing
ATL	Einstein notation + arbitrary indices + tensor construction + Iverson bracket	Rewrite rules
Exo	Imperative for-loops with limited control flow and array values	Hardware-aware rewrite rules; reconstructs Halide-like schedules
MDH	Pointwise + associative reductions + external operators (prefix sum)	Tiling / <code>compute_at</code> / <code>store_at</code> / parallelism
Fern	No explicit computation language	Halide-like scheduling model

Table 2.1: Comparison of tensor DSLs and scheduling models

2.2 Tiling

Loop tiling is an important optimization in code generation that improves parallelism and locality. Associative reductions are simple examples of functions that require all tiles to interact with each other to compute the final output. Some examples of associative reductions include computing the sum of the elements in an array, or the maximum of the elements in an array.

In Halide, the `rfactor` command is used to split an associative reduction into a two-part tiled function [1]. An example is shown in Figure 2.1. `rfactor` is defined further in Section 3.2. Outside of Halide, methods for tiling have been created that are specialized for a restricted set of operations. One example based on the theory of list homomorphisms [16] parallelizes a function over an array of numbers, requiring the user to define each input function recursively in multiple ways, both by appending and prepending elements to the array. Another example is *Compiling High Performance Recursive Filters* [4], which defines a domain-specific language in Halide for efficiently parallelizing linear filters. AutoLifter [2] is a compiler that uses search-based techniques to synthesize a merge function.

The Pochoir compiler [3] tiles stencil computations, which have multi-dimensional dependencies between array elements: the output at each timestep is a function of a window of

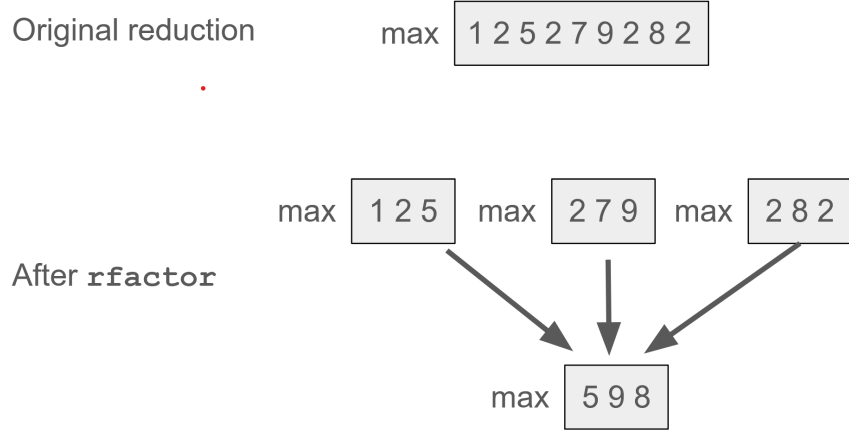


Figure 2.1: Example application of **rfactor**.

spatially local elements in the previous timestep. These complex dependencies make stencil computations difficult to tile.

2.3 Attention and FlashAttention

Attention [17] is a fundamental method used in machine learning. The attention algorithm takes three $N \times d$ matrices Q , K , V , where d is the dimensionality and N is the number of data points. The attention algorithm outputs $N \times d$ matrix O , where $O = \mathbf{softmax}(QK^T)V$.

To calculate O without computing each element of QK^T multiple times, every element in a given row must be stored in memory at the same time while the softmax of the row is calculated. Since attention is normally parallelized across rows, this direct approach requires a lot of memory.

The FlashAttention algorithm [18], our main inspiration for FlashTile, modifies standard attention by adding a layer of tiling within each row of QK^T . Thus, only a portion of a given row has to be stored in memory at once, significantly reducing the memory bandwidth of attention. To compute a single tile, the softmax operation is performed on a subset of the row, along with the product of a row of $\mathbf{softmax}(QK^T)$ and each column of V .

FlashAttention is a good example of an algorithm that involves nontrivial tiling. Figure 2.2 shows a high-level diagram of FlashAttention. The **softmax** operation in the definition of attention creates nontrivial dependencies where every element in a row of $\mathbf{softmax}(QK^T)$ depends on every other element in the same row, and yet FlashAttention is able to break up these dependencies to compute each row tile by tile. We can define ideas behind the construction of FlashAttention that are applicable to a broader class of algorithms. We define these ideas in Section 3, and apply these ideas to FlashAttention and other algorithms in Section 6.

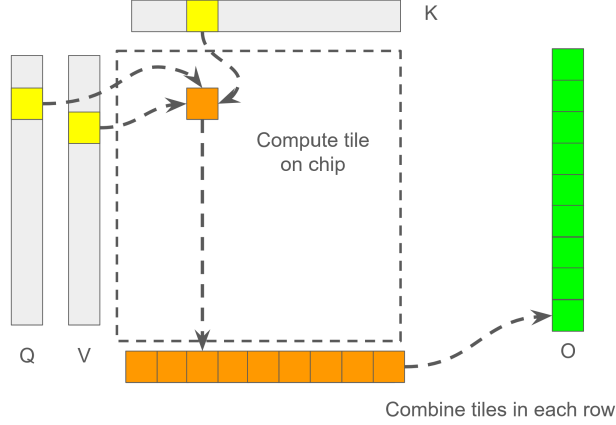


Figure 2.2: FlashAttention algorithm

2.4 Sliding windows

A sliding-window aggregation is an algorithm that computes a reduction over contiguous subarrays of a data stream, where each subarray contains the most recent elements in the stream. Figure 2.3 shows an example of a sliding window. Adjacent windows typically share most of their elements, potentially resulting in redundant work if each window is computed individually. Many methods have been proposed to reduce this redundant work. Pre-aggregation is a class of methods that involves computing *partial* aggregations, or *panes* [19], across small subsets of the array, so each window can be computed by combining the pre-computed partial results. Each partial can be used in multiple windows, potentially saving a lot of work.

The *bidirectional box-sum* algorithm [20] computes prefix sums and suffix sums in such a way that every fixed-size window can be computed by combining a prefix and a suffix. This algorithm is described in more detail in Section 3.3.

If the reduction operator has an inverse (for example, subtraction is the inverse of the addition operator), simpler and more efficient sliding window algorithms can be used [20]. One such algorithm is discussed in Section 3.3.

Other methods exist to make sliding-window aggregations more efficient, such as the *integral image* technique [21]. This technique involves performing a scan across an array to create a new array of prefixes. Any window can be calculated by subtracting one prefix from another. This technique requires the existence of an inverse function of the reduction operator.

2.5 Loop fusion

Recursively defined functions are common in algorithms involving arrays, such as complex image processing pipelines with recursive filters. These algorithms often perform multiple passes over data, which can be optimized using loop fusion. This is done in algorithms such as OpenCV’s depth perception algorithm [22]. FlashAttention [18] is a motivating example. FlashAttention computes several intermediate values for each tile, each of which have to

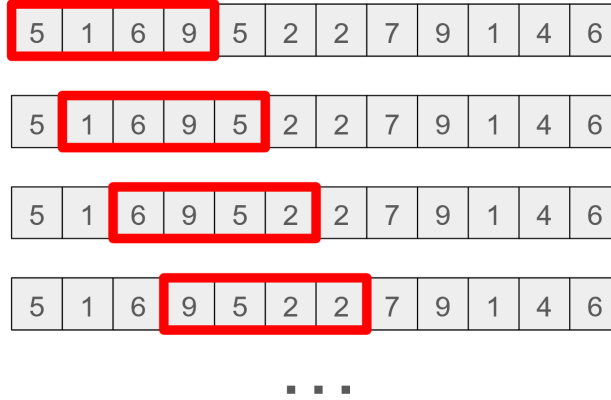


Figure 2.3: Sliding window algorithm

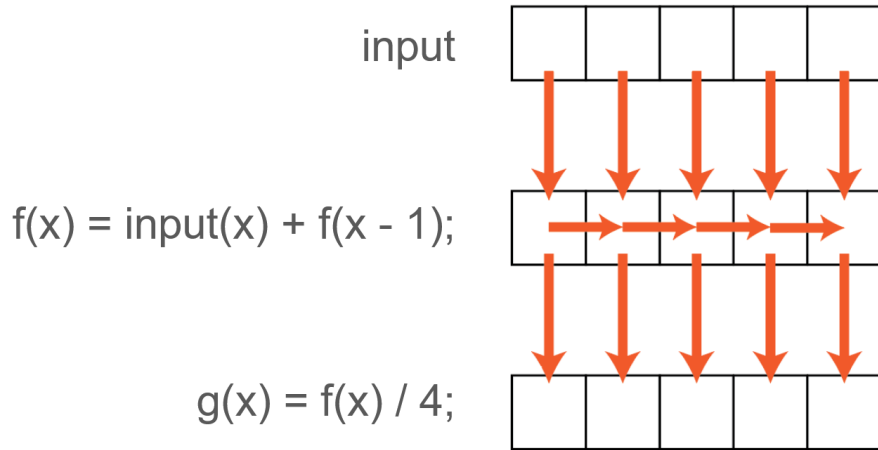


Figure 2.4: Dependency graph of a two-stage pipeline

be combined with intermediate values for other tiles in the same row. Loop fusion allows the intermediate values to be computed and used one tile at a time instead of computing intermediate values for an entire row before using any of them.

As a concrete example, consider a simple two-stage pipeline represented in Figure 2.4, where the first stage computes a prefix sum over an array, and the second stage divides each prefix sum element by 4. Without loop fusion, we need to compute the entire intermediate array before starting to compute the output, requiring two array passes and enough intermediate memory to store the entire prefix sum. With loop fusion, we can perform the division immediately after each prefix sum element is computed, so we only need a single array pass.

2.5.1 Loop fusion in Halide

Loop fusion is a prominent optimization in Halide. Halide can perform loop fusion in many programs involving pure functions. However, Halide is unable to perform loop fusion in loops involving reduction domains, which are Halide’s only method to express recursive functions.

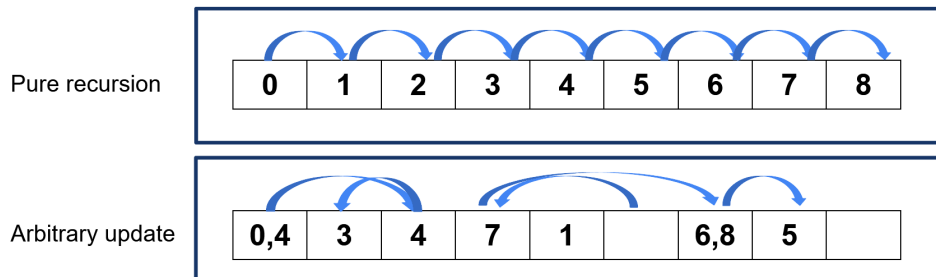


Figure 2.5: Comparison of a single-definition recursive function (top) and a more complicated update (bottom)

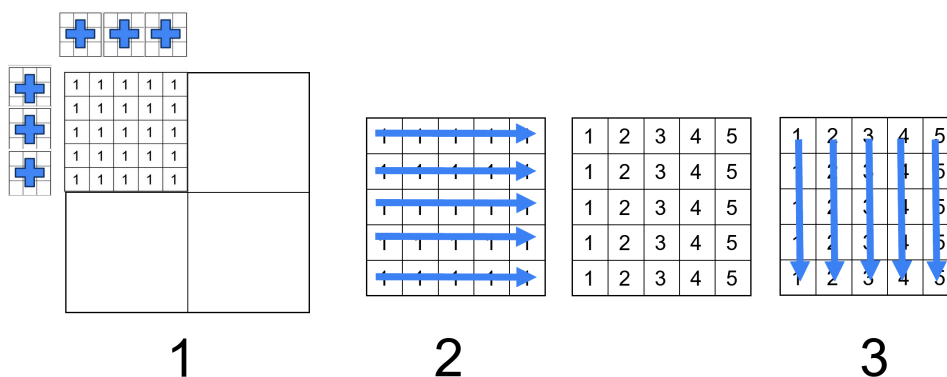


Figure 2.6: Simplified version of a poorly optimized box sum computation in large tiles. Step 1: Compute initial values at the edge of each tile. Step 2: Recursively compute 1-dimensional box sums by adding a new value and subtracting an old value at each step. Step 3: Recursively compute 2-dimensional box sums in the same manner.

In particular, Halide requires the function to iterate over all elements of the reduction domain before the function can be used. Loop fusion over recursive loops is difficult in general because the elements need to be computed in a certain order. In addition, as illustrated in Figure 2.5, Halide's implementation of reduction variables does not attempt to distinguish between recursively defined functions and arbitrary updates. Loop fusion is often possible for recursive functions but usually impossible in the latter case.

This Halide limitation has been known for years: Previous work has described how Halide was unable to express an optimal schedule for a depth-perception algorithm [22]. This algorithm efficiently computes several box sums centered on each pixel of the image by using a "sliding window" trick, performing recursion in multiple dimensions. Since the recursions cannot be interrupted, Halide can only express schedules that store large tiles in intermediate storage, computing box sums non-recursively at the edges of each tile. This is depicted in Figure 2.6. The OpenCV implementation, depicted in Figure 2.7, is much more memory-efficient, dividing the image into vertical strips and only storing a single row of each strip in intermediate storage.

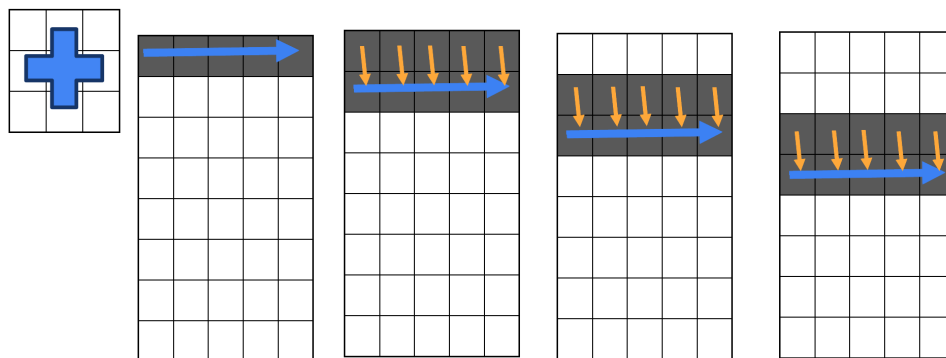


Figure 2.7: Dependency graph and memory storage (in gray) of OpenCV's StereoBM implementation at each loop iteration

Chapter 3

FlashTile theory

Scheduling languages such as Halide let programmers create a function definition and then modify how the function is computed, allowing programmers to test many kinds of optimizations without changing the function definition. However, many tiling-related optimizations require the function definition to be written in a decomposed form, as briefly described in Section 1. These optimizations are unusual for a scheduler because they require algorithmic changes, breaking the separation between function definition and function scheduling. We describe a framework of decompositions, showing the necessity of code transformations. Our framework focuses on tiling transformations that can only be performed when the function to be tiled has an algebraic decomposition that allows intermediate results to be combined into a final result.

3.1 Simple examples

We start by showing the `rfactor` scheduling command in Halide and comparing it to FlashTile, demonstrating how FlashTile is much more general than `rfactor` while sharing some high-level abstractions. Halide’s `rfactor` command can detect if a function is in the form of an associative reduction and, if so, transform it into a tiled form. However, `rfactor` is restricted to tiling functions that are explicitly written in the form of an associative reduction, while FlashTile can tile more complicated functions that can be transformed into associative reductions.

We provide a simple example of `rfactor` being performed on a simple summation. The original code can be written as shown in listing 3.1, which is equivalent to Algorithm 1. `rfactor` modifies the program of the `summation` function, making it equivalent to listing 3.2. With scheduling, the code executes Algorithm 2.

Listing 3.1: Application of rfactor

```

1 // input: A, tile_size
2 Var u;
3 RDom r(0, size);
4 RVar ro, ri;
5 summation() = 0;
6 summation() = summation() + A(r);
7 summation.update().split(r, ro, ri, tile_size);
8 summation.rfactor({ro, u});

```

Algorithm 1 Original summation

```

1: Inputs:  $A$  is an array with length  $N$ 
2: summation  $\leftarrow 0$ 
3: for  $j$  in range(0,  $N$ ) do
4:   summation  $\leftarrow$  summation +  $A[j]$ 
5: end for
6: Return summation

```

Listing 3.2: Transformed summation after rfactor

```

1 // inputs: A, tile_size
2 Var pure;
3 RDom r_tiled(0, tile_size), r2(0, size / tile_size);
4 summation_tiled(pure) = 0;
5 summation_tiled(pure) = summation_tiled(pure) + A(r_tiled + pure *
    tile_size);
6 summation() = 0;
7 summation() = summation() + summation_tiled(r2);

```

For comparison, we show a simple program that can be tiled using FlashTile but not using `rfactor`. This example involves a reduction over variable `r` that is used as input to another reduction over `r`. The following example is taken from the FuseMax paper [23], which contains the same scheduling transformations. Consider two vectors A and B with length N . Define $Y(A, B)$ as $\sum_{i=0}^{N-1} A[i]B[i]$ and $Z(A, B)$ as $Y(A, B) \cdot \sum_{i=0}^{N-1} A[i]$. In Halide, this can be represented as listing 3.3. This program executes Algorithm 3.

Listing 3.3: Original dot computation

```

1 // inputs A, B
2 RDom r(0, size);
3 Y() = sum(A(r) * B(r));
4 Z() = sum(Y() * A(r));

```

Algorithm 2 Transformed summation

```
1: Inputs:  $A$  is an array with length  $N$ 
2: for  $i$  in range(0, size/tile_size) do
3:   summation_tiled[ $i$ ]  $\leftarrow 0$ 
4:   for  $j$  in range(0, tile_size) do
5:     summation_tiled[ $i$ ]  $\leftarrow$  summation_tiled[ $i$ ] +  $A[j + i * \text{tile\_size}]$ 
6:   end for
7: end for
8: summation  $\leftarrow 0$ 
9: for  $I$  in range(0, size/tile_size) do
10:  summation  $\leftarrow$  summation + summation_tiled[ $I$ ]
11: end for
12: Return summation
```

Algorithm 3 Original dot algorithm

```
1: Inputs:  $A, B$  are both vectors with length  $N$ 
2: Outputs:  $Y(A, B) = \sum_{i=0}^{N-1} A[i]B[i]$ ,  $Z(A, B) = Y(A, B) \sum_{i=0}^{N-1} A[i]$ 
3: Initialize  $Y = 0$ ,  $Z = 0$ 
4: for  $i$  in range(0,  $N$ ) do
5:    $Y \leftarrow Y + A[i] \cdot B[i]$ 
6: end for
7: for  $i$  in range(0,  $N$ ) do
8:    $Z \leftarrow Z + Y \cdot A[i]$ 
9: end for
10: Return  $Z$ 
```

We could compute Y in its entirety before computing Z , but instead we wish to tile the combined computation of Y and Z in the i dimension, as in the FuseMax paper[23]. This is not possible with `rfactor`, which can only tile a single update step. Using FlashTile, this tiling is possible even though the original program is defined using two separate reductions. This tiled computation, shown in listing 3.4, is equivalent to Algorithm 4.

Listing 3.4: Tiled dot product computation

```

1 // inputs: A, B, size, tile_size
2 Var t;
3 RDom rtile(0, tile_size), r(1, size / tile_size - 1);
4 Y_tiled(t) = sum(A(rtile + t * tile_size) * B(rtile + t * tile_size));
5 Z_tiled(t) = sum(Y_tiled(t) * A(rtile + t * tile_size));
6 merged_tile() = Tuple(Y_tiled(0), Z_tiled(0));
7 Expr merge_y = merged_tile()[0] + Y_tiled(r);
8 Expr merge_z = merge_y * (merged_tile()[1] / merged_tile()[0] + Z_tiled(r)
    ) / Y_tiled(r));
9 merged_tile() = Tuple(merge_y, merge_z);
10 Z() = merged_tile()[1];

```

Algorithm 4 Tiled dot algorithm

```

1: Inputs:  $A, B$  are both vectors with length  $N$ 
2: Outputs:  $Y(A, B) = \sum_{i=0}^{N-1} A[i]B[i]$ ,  $Z(A, B) = Y(A, B) \sum_{i=0}^{N-1} A[i]$ 
3: Initialize  $Y = 0, Z = 0$ 
4: Divide  $A$  into  $K$  blocks  $A_0, \dots, A_{K-1}$  of size  $N/K$ 
5: Divide  $B$  into  $K$  blocks  $B_0, \dots, B_{K-1}$  of size  $N/K$ 
6: for  $i$  in range(0,  $K$ ) do
7:    $\tilde{Y} \leftarrow 0$ 
8:   for  $j$  in range(0,  $N/K$ ) do
9:      $\tilde{Y} \leftarrow \tilde{Y} + A_i[j] \cdot B_i[j]$ 
10:  end for
11:   $\tilde{Z} \leftarrow 0$ 
12:  for  $j$  in range(0,  $N/K$ ) do
13:     $\tilde{Z} \leftarrow \tilde{Z} + \tilde{Y} \cdot A_i[j]$ 
14:  end for
15:   $Y' \leftarrow Y + \tilde{Y}$ 
16:  if  $i = 0$  then
17:     $Z \leftarrow \tilde{Z}$ 
18:  else
19:     $Z \leftarrow Y' \cdot ((Z/Y) + (\tilde{Z}/\tilde{Y}))$ 
20:  end if
21:   $Y \leftarrow Y'$ 
22: end for
23: Return  $Z$ 

```

3.2 Description of transformations

We continue by describing the transformations used by `rfactor` and FlashTile, providing a foundation for the generality of FlashTile.

Definition 3.1. We define function $f : P(X) \rightarrow X$ as an *associative reduction*, where P is the powerset function, if for any partition of input set A into subsets $A_0, \dots, A_{k-1}$, we have

$$f(A) = f(\{f(A_0), \dots, f(A_{k-1})\}).$$

`rfactor` performs the above factorization on an associative reduction, dividing the input array into equal-sized tiles that can be processed independently. `rfactor` also modifies the definition of the input function. The transformation in Definition 3.1 is an algebraic decomposition of the original function that depends on the function satisfying the associative property. It is feasible for a compiler such as Halide to identify that a function is associative, and so this transformation is possible in many cases. The methods behind this associativity check are detailed in the `rfactor` paper [1].

FlashTile uses a more general class of transformations, which we describe here. Let $f : P(X) \rightarrow Y$ be a function mapping sets (with elements from domain X) to values in domain Y . For an input set $A \subseteq X$, we wish to decompose A into disjoint subsets $A_0, \dots, A_{k-1}$ such that

$$B_i = f(A_i) \quad \text{for all } i \in \{0, \dots, k-1\},$$

and

$$f(A) = m(B_0, \dots, B_{k-1})$$

for some *merge function* $m : Y^* \rightarrow Y$. This factorization is similar to `rfactor`, the difference being that `rfactor` uses f to merge tiles together, while our framework uses an arbitrary merge function m . We apply this idea to optimize two classes of algorithms: reductions and sliding windows. For each class of algorithms, FlashTile uses a pre-defined algorithmic template that is used to efficiently combine tiles. Each template uses a merge function that is constructed at compilation time.

Sliding windows

Sliding window functions compute many overlapping reductions, allowing FlashTile to perform optimizations that take advantage of redundant work. We extend the merge function framework to describe optimizations for sliding-window aggregations. In this section, we use ordered arrays instead of unordered sets.

Definition 3.2. Given an array A , a function $g(t)$ is a *sliding-window aggregation* if it equals $f(A[t:s : t:s + w])$ for some constants s and t , where s is the window slide and w is the window length.

For the problems we consider, the main benefit of tiling sliding windows is a reduction in total work. If s divides w , and f has an associated merge function m , then a sliding-window aggregation can be tiled using the following algorithm:

Define $A'_i = A[i \cdot s : (i + 1) \cdot s]$. Each output window $g(t)$ can be computed by merging w/s partial windows of the form $f(A'_i)$. Specifically,

$$g(t) = f(A[ts : ts + w]) = m(f(A'_t), \dots, f(A'_{t+(w/s)-1})).$$

We show an example of a sliding window computation that FlashTile can optimize. First, we write a summation in Halide as shown in listing 3.5. The algorithmic representation of this summation is in Algorithm 5. The optimized algorithm in Halide, shown in listing 3.6, is much more complex. This code is equivalent to Algorithm 6. This algorithm, a bidirectional box-sum, is discussed in Section 3.3.

Listing 3.5: Sliding window summation

```

1 // inputs: A, window
2 // output: sliding window summation over A
3 Var t;
4 RDom r(0, window);
5 out(t) = 0;
6 out(t) = out(t) + A(t + r);

```

Algorithm 5 Original sliding window computation

```

1: Inputs:  $A$  is an array with length  $N$ 
2: for  $j$  in range(0,  $N - w$ ) do
3:    $\text{out}[j] \leftarrow 0$ 
4:   for  $k$  in range(0,  $w$ ) do
5:      $\text{out}[j] \leftarrow \text{out}[j] + A[j + k]$ 
6:   end for
7: end for
8: Return out

```

Listing 3.6: Optimized sliding window summation

```

1 // inputs: A, window
2 // output: sliding window summation over A
3 Var t, i, pure;
4 RDom r(1, window - 1);
5 prefix(t, pure) = 0;
6 suffix(t, pure) = 0;
7 prefix(0, pure) = A(window * (pure + 1));
8 suffix(window - 1, pure) = A(window * (pure + 1) - 1);
9 prefix(r, pure) = prefix(r - 1, pure) + A(window * (pure + 1) + r);
10 suffix(window - 1 - r, pure) = A(window * (pure + 1) - 1 - r) + suffix(
    window - r, pure);
11 out(i) = select(i % window == 0, suffix(0, i / window), prefix((i - 1) %
    window, i / window) + suffix(i % window, i / window));

```

Algorithm 6 Optimized sliding window computation

```
1: Inputs:  $A$  is an array with length  $N$ 
2: prefix and suffix are 2D arrays with dimensionality  $(w, N/w + 1)$ 
3: Outputs:  $\text{out}[i] = \sum_{k=0}^{w-1} A_{i+k}$ 
4: for  $k$  in range(0,  $N/w$ ) do
5:   prefix[0,  $k$ ]  $\leftarrow A[w \cdot (k + 1)]$ 
6:   suffix[ $w - 1$ ,  $k$ ]  $\leftarrow A[w \cdot (k + 1) - 1]$ 
7:   for  $i$  in range(1,  $w$ ) do
8:     prefix[ $i$ ,  $k$ ]  $\leftarrow \text{prefix}[i - 1, k] + A[w \cdot (k + 1) + i]$ 
9:     suffix[ $w - 1 - i$ ,  $k$ ]  $\leftarrow A[w \cdot (k + 1) - 1 - i] + \text{suffix}[w - i, k]$ 
10:  end for
11: end for
12: for  $k$  in range(0,  $N/w$ ) do
13:   out[ $w \cdot k$ ]  $\leftarrow \text{suffix}[0, k]$ 
14:   for  $i$  in range(1,  $w$ ) do
15:     out[ $w \cdot k + i$ ]  $\leftarrow \text{suffix}[i, k] + \text{prefix}[i - 1, k]$ 
16:   end for
17: end for
18: Return out
```

3.3 Algorithms for processing tiles

We now fully describe how we use decomposed functions.

Definition 3.3. Given functions $\text{inv} : Y \rightarrow P(X)$ and $f : P(X) \rightarrow Y$, we define a *decomposed merge function* $m(B_0, \dots, B_{k-1})$ as $f(\text{inv}(B_0), \dots, \text{inv}(B_{k-1}))$.

Informally, the function inv processes the outputs of each tile and f combines the outputs. This definition allows $f(A)$ to be computed using Algorithm 7. This algorithm is a fundamental part of the FlashTile framework because it transforms the most basic operation we consider, a reduction, into a tiled form.

Algorithm 7 Reduction algorithm

```
1: Inputs:  $K$  values  $A_0, \dots, A_{K-1}$ 
2: out  $\leftarrow f(A_0)$ 
3: for  $J$  in range(1,  $K$ ) do
4:   out  $\leftarrow f(\text{inv}(\text{out}), \text{inv}(f(A_J)))$ 
5: end for
6: Return out
```

We detail the algorithms that we use to optimize the calculation of sliding windows, which are defined in Section 3.2.

Bidirectional box-sum The partial-based sliding-window algorithm in Section 3.2 involves combining w/s partials for each window, which can be costly if w is much larger than s . The bidirectional box-sum (BDBS) algorithm, defined in Algorithm 8, is one method to efficiently calculate sliding windows [20]. Although it can be used to calculate multi-dimensional windows, in our case we restrict it to one dimension. Using BDBS, the number of merge operations used to calculate each window is an amortized constant regardless of w and s . In the following description, we assume $s = 1$. The algorithm can extend to arbitrary s , assuming that s divides w , since we can precompute partials using the above algorithm and run BDBS with the array of partials as input.

Algorithm 8 Bidirectional box-sum

```

1: Inputs:  $A$  is an array with length  $N$ 
2: prefix and suffix are 2D arrays with dimensionality  $(w, N/w + 1)$ 
3: Outputs:  $\text{out}[i] = \bigoplus_{k=0}^{w-1} A_{i+k}$ 
4: for  $k$  in range(0,  $N/w$ ) do
5:    $\text{prefix}[0, k] \leftarrow A[w \cdot (k + 1)]$ 
6:    $\text{suffix}[w - 1, k] \leftarrow A[w \cdot k + w - 1]$ 
7:   for  $i$  in range(1,  $w$ ) do
8:      $\text{prefix}[i, k] \leftarrow \text{prefix}[i - 1, k] \bigoplus A[w \cdot (k + 1) + i]$ 
9:      $\text{suffix}[w - 1 - i, k] \leftarrow A[w \cdot k + w - 1 - i] \bigoplus \text{suffix}[w - i, k]$ 
10:  end for
11: end for
12: for  $k$  in range(0,  $N/w$ ) do
13:    $\text{out}[w \cdot k] \leftarrow \text{suffix}[0, k]$ 
14:   for  $i$  in range(1,  $w$ ) do
15:      $\text{out}[w \cdot k + i] \leftarrow \text{suffix}[i, k] \bigoplus \text{prefix}[i - 1, k]$ 
16:   end for
17: end for
18: Return out

```

Difference-based sliding windows Using what we term *difference functions*, it is possible to compute sliding windows more efficiently than purely merge-based methods such as bidirectional box-sum, but their usage is more limited than these merge-based methods because many merge functions do not have an associated difference function.

Definition 3.4. A *difference function* with respect to f is a function d that satisfies $d(f(A), f(B)) = f(A \setminus B)$ for all inputs A, B where $B \subset A$.

For example, if $f(A)$ adds the elements in A , then we can define $d(x, y) = x - y$. The existence of a merge function for f does not guarantee the existence of a difference function; for example, if $f(A)$ outputs the maximum element in A .

For a 1-dimensional array A , define the window sum W_x as $\bigoplus_{k=0}^{w-1} f(A_{x+k})$, where $\bigoplus$ is a merge function. Then $W_{x+1} = d(W_x \oplus A_{x+w-1}, A_x)$, where w is the number of elements in the window, and d is a difference function. That is, W_{x+1} can be computed from W_x using only two operations.

3.4 Derivation of merge functions

Here, we provide some analysis of merge functions that sheds some light on how FlashTile can efficiently construct them. We reduce the problem of constructing the merge function to the problem of constructing an inverse function.

Definition 3.5. A merge function $m : Y \times Y \rightarrow Y$ satisfies the *merge property* with respect to f if $f(A \cup B) = m(f(A), f(B))$ for all sets A and B .

Definition 3.6. A function inv satisfies the *weak inverse property* with respect to f if, for all A , $f(A) = f(\text{inv}(f(A)))$.

It can be shown that if m satisfies the merge property, then

$$m(f(A), f(B)) = f(\text{inv}(f(A)), \text{inv}(f(B)))$$

for all x in the image of f , where inv is an arbitrary function that satisfies the weak inverse property. This is proven in Proposition A.1. This proposition provides a powerful way to construct a merge function: it is only necessary to find a weak inverse of the function f . Assuming this inverse can be found, it is only necessary to determine whether a merge function exists. This problem is discussed in Section 3.5. Techniques to process definitions in the source code are discussed in Section 4.2.

Finally, we use the definition of a merge function to construct a difference function. To our knowledge, this construction is novel. Assume $d(X, Y)$ is of the form $f^-(\text{inv}(X), \text{inv}(Y))$. As before, we assume that inv is a weak inverse of f . We require f^- to have two elements, since the difference function should only take the difference of two sets. We show in Proposition A.2 that if a merge function exists, and f has an inverse g in its second element that satisfies $g(f(x, y), y) = x$, then f^- can be constructed using f and g .

3.5 Merge function existence

For what functions can we guarantee that a merge function exists? This is difficult to answer in general, but we describe a few classes of functions to guide the focus for the set of algorithms FlashTile can handle.

A merge function always exists for an associative reduction because these reductions can be tiled with the **rfactor** transformation. A concatenation of independent associative reductions is also an associative reduction itself, so the concatenation can be tiled with the **rfactor** transformation. The **rfactor** paper describes how **rfactor** can tile these concatenations [1].

If f is an invertible function of one or more associative reductions, the values of the reductions can be uniquely computed from a tile of f . Since the associative reductions can be computed from f , it is possible to create a merge function for f .

3.6 Dot product example

To demonstrate our framework, we apply the above concepts to explain how FlashTile transforms the computation in Section 3.1, shown in listing 3.3. The main difficulty is constructing a merge function to combine tiles of $Z(A, B)$.

To define a merge function for $Z(A, B)$, we first note that $Z(A, B)$ can be defined as a function of two associative reductions:

$$Z = \left(\sum_{i=0}^{N-1} A[i]B[i] \right) \left(\sum_{i=0}^{N-1} A[i] \right).$$

Define $X(A, B)$ as a vector of 2-element tuples, where $X[i] = \{A[i]B[i], A[i]\}$. Then we may define associative reductions $r_0(X) = \sum_{i=0}^{N-1} X[i][0]$, $r_1(X) = \sum_{i=0}^{N-1} X[i][1]$. We then define $g(x, y) = \{x \cdot y, y\}$, where $x \cdot y$ corresponds to Z and y corresponds to Y . Note that $g^{-1}(x, y) = \{x/y, y\}$. So, we define $f(X)$ as $g(r_0(X), r_1(X))$.

Let the input tiles for the merge function be $\{x_L, y_L\}, \{x_R, y_R\}$. We pass both tiles through g^{-1} and apply f to the concatenation, resulting in merge function

$$\begin{aligned} & f(\{g^{-1}(x_L, y_L), g^{-1}(x_R, y_R)\}) \\ &= f(\{(\frac{x_L}{y_L}, y_L), (\frac{x_R}{y_R}, y_R)\}) \\ &= g(r_0(\frac{x_L}{y_L}, y_L), r_1(\frac{x_R}{y_R}, y_R)) \\ &= g(\frac{x_L}{y_L} + \frac{x_R}{y_R}, y_L + y_R) \\ &= \{(y_L + y_R)(\frac{x_L}{y_L} + \frac{x_R}{y_R}), y_L + y_R\}. \end{aligned}$$

Using this function, we compute Z using Algorithm 4.

Chapter 4

FlashTile infrastructure

In this chapter, we reconcile our theory to the reality of Halide. FlashTile in Halide must handle Halide’s representation of computation (forests of functions and stages [24]) and the scheduling system. Despite these difficulties, we show how the idea of FlashTile can be realized in a modular and extensible manner. Unlike prior work on automatically tiling functions, we show that separating the components of the tiling transformations has virtue because they can be extended to handle new cases. FlashTile’s merge/inverse function synthesis is separated from how we use the merge function, allowing us to propose multiple algorithms that utilize merge functions. We also have a third modular component to remedy the issues of applying this theory to Halide.

4.1 Scheduling preconditions and recognizing functions in Halide

So far, our theory works on abstracted reduction functions, but in Halide the abstract function that we want might be a call tree of Halide functions, a sub-tree of the forest of Halide functions representing a Halide program. (Halide functions are nodes in the forest while edges represent produce consumer relationships (or simply calls to other function from functions and their stages).) Additionally, our scheduling command might disrupt the scheduling system on some functions in this forest. To remedy these issues, we allow users to specify an output function and the intermediate functions that make up the tree that we wish to extract as our abstract reduction function. The root of the sub-tree provides the output of the abstract function. The inputs to the abstracted function are the function calls at the leaves of the tree. The extraction and preparation of this tree is described more thoroughly in appendix section A.2. We also require that, at the location in a Halide program where FlashTile is applied, no Halide scheduling commands have previously been applied to any function in this tree. The exception is the output, which must be realized somewhere and otherwise unscheduled except for tiling. Though this might be overly restrictive, it is sufficient to ensure that we don’t break the scheduling system.

4.2 Merge functions and their synthesis

All of our transformed algorithms use a merge or inverse function. We must represent these for use in these algorithms, so users can specify them or so we can synthesize them automatically. In the FlashTile framework, a merge function is represented as a tuple of expressions with free variables. Each free variable is substituted with an output from one of the two tiles to be combined. The merge function cannot be implemented as a Halide function because the inputs to the merge function are not necessarily in Z^n .

Merge function synthesis

For completeness, FlashTile also contains a method `do_merge` to generate a merge function. It is also possible for programmers to manually write a merge function. We provide a potential implementation here that is well-suited to the types of algorithms found in Halide.

`do_merge` takes as input a function f , which contains a call tree to construct a merge function for, along with a reduction variable r over which f should be tiled. We represent this restriction with a simplified grammar as the input:

$$\begin{aligned} \text{expr} &::= \text{const} \mid \text{var} \mid \text{binop}(\text{expr}, \text{expr}) \\ &\quad \mid \text{unop}(\text{expr}) \mid \text{reduction}(\text{op}, \text{expr}, \text{var}) \\ \text{binop} &::= + \mid - \mid \times \mid / \\ \text{unop} &::= \text{exp} \mid - \\ \text{reduction} &::= \text{max} \mid \text{min} \mid \text{sum} \end{aligned}$$

`do_merge` outputs a tuple of expressions that corresponds to the output of merge function m . Examples of the transformations performed by `do_merge` can be seen in the Theory chapter under Section 3.1. `do_merge` is implemented through recursive pattern matching on rewritten expressions. For example, on this simplified grammar we apply the distributive property before matching:

$$\begin{aligned} (\text{Distributive property}) \quad &\text{reduction}(+, a \times b, v), \quad v \notin \text{vars}(a) \\ &\longrightarrow a \times \text{reduction}(+, b, v) \end{aligned}$$

Here, $\text{vars}(a)$ represents the set of all variables in the expression tree of a . $+$ and $\times$ can be replaced by any pair of operations that satisfy the distributive property. This transformation is useful because it allows for chains of nested reductions to be broken up into simpler reductions that are easier to synthesize merge functions for. We pattern-match on expressions in the grammar to find simple weak inverses to combine into a single larger weak inverse that can be used in the various merge function formulas in Theory section 3.4. For example, if we run into $v + a$ where a is a constant and v is a variable, we have a full inverse. Here, we provide a more general mechanism for this construction as a recursive operation on the grammar, assuming we apply rewrites before each match:

$$\text{inv}_v(e) = \begin{cases} (\{0\}, \{e\}) & \text{if } v \notin \text{vars}(e) \\ (\{e\}, \{e\}) & \text{if } e = \text{reduction}(\text{op}, e_2, v) \\ & \text{and } v \notin \text{vars}(e_2) \\ \text{combine}(\text{inv}_v(e_1), \text{inv}_v(e_2)) & \\ \text{.replace_output}(e_1, \text{op}^{-1}(\text{op}(e_1, e_2), e_2)) & \\ \text{.replace_input}(e_1, \text{op}(e_1, e_2)) & \text{if } e = \text{op}(e_1, e_2) \end{cases}$$

`inv` outputs a list of inputs and a list of outputs that form the weak inverse. The `combine` function concatenates the expressions in its arguments. The `replace_output` and `replace_input` functions modify all outputs and inputs in the given lists. Finally, the grammar includes 0 because it assumes operations have a valid identity.

Difference merge functions

FlashTile extends `do_merge` to derive a difference function (defined in Section 3.3). Since weak inverses can be used in both merge functions and difference functions, the implementation only requires a few modifications to `do_merge`. When computing a difference function, the main changes to `do_merge` are that many of the arithmetic operations in the outputted merge function are swapped with their inverses. The max and min reduction operations are unable to be tiled this way, since their binary operators do not have inverses.

4.3 Combining tiles

After users have extracted a function and found a merge function, they can choose from several ways to transform the function, each of which is an algorithm in Section 3. These different methods allow FlashTile to optimize a variety of algorithm classes, from non-associative reductions to sliding-window aggregations. After performing the transformation, FlashTile provides handles to the various transformed intermediates in the tree and the algorithm-specific merge step so they can be further scheduled by the user.

Descriptions of the resulting code can be found in listing 4.1, listing 4.2, listing 4.3, and listing 4.4, and are explained further here.

- **Basic reductions:** The simplest algorithm we support is a reduction that merges all tiles along the same dimension. The algorithm defined in listing 4.1 creates a function `intm(x, t)` that represents the output of each tile, where `t` is the tile index. The tiles are then combined using a merge function. After the tiles are combined, any auxiliary functions used only in the merge stage are discarded. This combination step outputs a tuple of values, where the first element in the tuple is the desired output of the original function and the other elements are auxiliary functions which can be discarded. The algorithm defined in listing 4.2, which computes a basic sliding-window computation, is a simple variant of listing 4.1 that creates one output for every tile index `t` instead of just one output across all tile indices.

- **Bidirectional box-sum:** The bidirectional box-sum algorithm, shown in listing 4.3, is implemented using several stages. In the first stage, `ft` and auxiliary functions are computed for each tile. The next stages use a scan algorithm to compute prefixes and suffixes; this is done by combining the tiles using the merge function. The last stage computes each window by merging a prefix and a suffix.
- **Difference-based sliding window:** The difference-based sliding window aggregation algorithm, shown in listing 4.4, has three stages. In the first stage, `ft` and auxiliary functions are computed for each tile. In the second stage, the window at the beginning of each tile is computed. In the third stage, it recursively computes the rest of the windows in each tile using merge function `merge_expr` and difference function `inverse_expr`.

The process of transforming the code is similar to the steps described in [1], though potentially performed on a call tree. Our implementation starts from that of the `rfactor` command, reusing that infrastructure as much as possible to build more complex transformations of Halide code. This makes sense as the main mechanical operations are similar: rewriting definitions to use tiled variables and adding new definitions/stages to use merge functions.

Listing 4.1: Basic reduction

```

1 // inputs: ft, gt1, ..., gtk, size, tileSize
2 Var x, t;
3 RDom r(1, size / tileSize - 1);
4 intm(x, t) = Tuple(ft(x, t), gt1(x, t), ..., gtk(x, t));
5 out_tuple(x) = intm(x, 0);
6 out_tuple(x) = merge_expr.subst(out_tuple(x), intm(x, r));
7 f(x) = out_tuple(x)[0];

```

Listing 4.2: Basic sliding window algorithm

```

1 // inputs: f, merge_expr, ft, gt1, ..., gtk, winsize, tileSize
2 Var x, t;
3 RDom r(1, winsize / tileSize - 1);
4 intm(x, t) = Tuple(ft(x, t), gt1(x, t), ..., gtk(x, t));
5 out_tuple(x, t) = intm(x, t);
6 out_tuple(x, t) = merge_expr.subst(out_tuple(x, t), intm(x, t + r));
7 f(x, t) = out_tuple(x, t)[0];

```

Listing 4.3: Bi-directional box sum algorithm

```

1 // inputs: ft, gt1, ..., gtk, winsize, tileSize
2 Var x, t, t2;
3 Expr wt = winsize / tileSize;
4 RDom r(1, wt - 1);
5 intm(x, t) = Tuple(ft(x, t), gt1(x, t), ..., gtk(x, t));
6 prefix(x, t, t2) = Tuple(0, ..., 0);
7 prefix(x, t, 0) = intm(x, t * wt);
8 suffix(x, t, t2) = Tuple(0, ..., 0);
9 suffix(x, t, wt - 1) = intm(x, t * wt + wt - 1);
10 prefix(x, t, r) = merge_expr.subst(prefix(x, t, r - 1), intm(x, t * wt +
    r));
11 suffix(x, t, wt - 1 - r) = merge_expr.subst(suffix(x, t, wt - r), intm(x,
    (t + 1) * wt - 1 - r));
12 out_tuple(x, t2) =
13     select(t2 % wt == 0, suffix(x, t2 / wt, 0),
14         merge_expr.subst(suffix(x, t2 / wt, t2 % wt),
15             prefix(x, (t2 / wt) + 1, (t2 - 1) % wt)));
16 out(x, t2)=out_tuple(x, t2)[0];

```

Listing 4.4: Difference-based sliding window algorithm

```

1 // Inputs: Tiled function ft, gt1, ..., gtk, winsize, tileSize, N
2 Var x, t;
3 RDom r(1, winsize / tileSize - 1), r2(1, N / tileSize - 1);
4 intm(x, t) = Tuple(ft(x, t), gt1(x, t), ..., gtk(x, t));
5 out_tuple(x, t) = Tuple(0, ..., 0);
6 out_tuple(x, 0) = intm(x, 0);
7 out_tuple(x, 0) = merge_expr.subst(out_tuple(x, 0), intm(x, r));
8 out_tuple(x, r2) =
9     inverse_expr.subst(
10         merge_expr.subst(out_tuple(x, r2 - 1), intm(x, r2)),
11         intm(x, r2 - winsize / tileSize));
12 f(x, t) = out_tuple(x, t)[0];

```

We examine listing 4.2, a transformed sliding window algorithm, to see this more clearly. Given the call tree of functions rewritten to use tiled variables and some metadata about a sliding window function, an intermediate function is formed. Then, given the merge function expression, the intermediate tuples are substituted into the merge expression to make an output that is patched into the original output function. Except for the small details of checking that we found a sliding window and providing all the handles, this completes the transformation.

4.4 Modifications to Halide

In the process of creating FlashTile, we made several modifications to the Halide compiler. These modifications addressed various inefficiencies that would otherwise slow down the functions produced by FlashTile.

Contextual variables in predicates

Halide often overcomputes: it sometimes computes arrays with many values that are never used later in the program, and it also sometimes redundantly computes outputs at points that were computed in previous loop iterations. In fact, recomputation is described as a tradeoff that Halide needs to balance with parallelism and locality [5]. Although the Halide compiler performs a *sliding-window optimization* to eliminate redundant computations, this optimization is only performed if the window boundaries slide in one dimension. Unfortunately, one of our experiments involves a sliding window whose boundaries slide in two dimensions. It could be possible for Halide to detect this redundant computation using better bounds inference, although it would require complicated infrastructure. Instead, we implement a more general transformation.

We extend the usage of predicates in Halide to get around the problem of overcomputation. Halide supports adding a predicate to a reduction domain to limit the domain’s scope. This predicate is then bound to any stage where the domain is used. For example, given reduction domain $\mathbf{r}$, a Halide program could add the predicate $\mathbf{r} < \mathbf{x}$ and use $\mathbf{r}$ in a definition where $\mathbf{x}$ is also used. We eliminate unnecessary computation by adding predicates to the relevant functions at the user level. However, we needed to modify the compiler to allow Halide to compile these predicates.

The Halide compiler requires that any variable used in a predicate also appears in the function stage that contains it. Otherwise, the output could depend on the schedule, undermining Halide’s separation between definitions and schedules. In one of our experiments, however, the desired schedule computes non-rectangular regions of FlashTile’s intermediate functions that vary over time, and the time variable does not appear in the definitions. Predicates naturally describe these regions, but here they reference a variable absent from the corresponding functions. To support this, we modified Halide to allow predicates to use such contextual variables. During compilation, we detect them and search the surrounding loop nest to associate them with a loop variable.

Loop-invariant code motion

Another weakness of the Halide compiler is its loop-invariant code motion. This optimization detects when an expression is redundantly computed during every iteration of a loop, hoisting the expression so that it is computed only once outside of the loop. However, the Halide compiler fails to recognize when a load operation is loop-invariant, instead assuming that no loads can be lifted out of the loop. This failure can cascade to expressions in the loop that depend on the loaded value, potentially causing expensive computations to be redundantly performed inside of the loop when they are actually loop-invariant.

We modify the compiler so that it attempts to hoist these loads under certain conditions where it is guaranteed to be safe. Specifically, it checks that there are no stores to the same location in the same loop.

4.5 Problems with FlashAttention

Our FlashAttention experiments in Section 6.1 do not include direct runtime comparisons on a GPU. After several attempts to write an optimized FlashAttention program in Halide, we found that doing so was infeasible. A fundamental problem with implementing FlashAttention in Halide is that it requires loop fusion for recursively defined intermediate functions, and Halide restricts how recursive functions can be scheduled. The FlashAttention algorithm computes several values that are updated once every time a tile is processed: m_i , l_i , and O_i . FlashAttention uses intermediate values of m_i and l_i to update O_i , and also uses intermediate values of m_i to update l_i .

Difficulties with writing FlashAttention directly. Because the variables m_i , l_i , and O_i are updated several times, their definitions can only be represented in Halide by recursively defined functions. This requires the use of RDom that iterate over the tiles. However, Halide restricts how iteration over RDom can be scheduled: if a function definition contains an RDom, it must iterate over the RDom’s entire iteration space before the function can be used to compute another function. In the case of FlashAttention, this restriction implies that all intermediate values of m_i and l_i in a row of tiles must be computed and stored before they can be used to compute O_i for any tile in the same row. This uses a lot of unnecessary memory and requires the program to perform multiple passes over each tile, resulting in worse performance than the actual FlashAttention algorithm.

Problems with attempted workarounds. It is possible to partially work around this restriction, but not in its entirety. Halide can perform each update of m_i and l_i at the same time with a single RDom by defining a 2-tuple-valued function that contains both m_i and l_i . One might ask whether it is possible for Halide to fully emulate FlashAttention by defining a 3-tuple-valued function containing m_i , l_i , and elements of O_i . It is more difficult to extend this workaround to O_i because O_i has one more dimension than m_i and l_i ; O_i can be updated in parallel over this extra dimension. Each 3-tuple must contain one value of m_i and l_i for each element of O_i , even though only one value of m_i and l_i is needed per *row* of O_i . Nevertheless, there are multiple ways to schedule a 3-tuple-valued function containing m_i , l_i , and O_i that performs in-place updates in a single pass instead of fully computing and storing all intermediate values of m_i and l_i , but none of them match the original schedule of FlashAttention. One way is to create a pure variable that covers the extra dimension of O_i , but this requires every single 3-tuple to separately compute its own value of m_i and l_i , which is computationally expensive. One can avoid this expensive computation by computing m_i and l_i in exactly one of the row’s 3-tuples, then using those values of m_i and l_i to update every other element of O_i in the row. This approach requires using a RVar to iterate over the extra dimension of O_i , stopping the updates from being performed in parallel over this dimension.

Chapter 5

Inductive functions

Based on incomplete efforts to implement FlashAttention in Halide, our efforts shifted to creating a principled solution by adding a new feature to the Halide compiler, which is called *inductive functions*. This feature uses recursive functions to allow Halide to express many schedules that cannot otherwise be expressed. Inductive functions follow Halide’s design philosophy and are designed to work with the rest of Halide’s features. To our knowledge, this concept has not been implemented previously in any scheduling language. Although we have not yet used inductive functions to implement FlashAttention, preliminary experiments show that inductive functions can result in more performant schedules.

5.1 Construction and implementation

We go over how inductive functions are defined and how they can be used to perform loop fusion. Each portion of the inductive function implementation corresponds to a step in Halide’s compilation pipeline, which is visualized in Figure 5.1.

5.1.1 Definition

We describe the syntax and semantics of inductive functions in Halide. This corresponds to the **Definition** stage in the compilation pipeline in Figure 5.1. An inductive function in Halide is a pure function with a recursive definition; that is, the function definition calls previously computed values of itself. To define it formally:

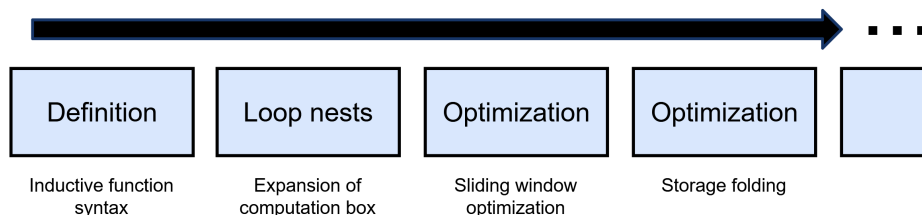


Figure 5.1: Halide’s compilation pipeline. The boxes describe general steps in Halide’s compilation process. The text below each box describes compilation steps that are specific to inductive functions.

Definition 5.1 (Inductive function). A Halide inductive function is a function $f : Z^n \rightarrow E$ defined by:

$$f(x_1, \dots, x_n) = \text{select}(r(x_1, \dots, x_n), e^{\text{base}}, e^{\text{rec}}),$$

where $\text{select}(c, e_1, e_2)$ evaluates to e_1 if c is true and to e_2 otherwise, and:

- $r : Z^n \rightarrow \{\text{true}, \text{false}\}$,
- $e^{\text{base}} \in E$ contains no occurrences of f ,
- $e^{\text{rec}} \in E$ contains at least one occurrence of f .

We require inductive functions to be defined using the `select()` method, which divides the recursion into a base case and a recursive case. This construct fits well into Halide’s design philosophy where most indices in function definitions are pure variables.

Let m be the number of syntactic occurrences of f in e^{rec} . Each occurrence is of the form:

$$f(s_j(x_1, \dots, x_n)), \quad j \in \{1, \dots, m\},$$

where $s_j : Z^n \rightarrow Z^n$.

Definition 5.2 (Termination over r). Let $\{s_1, \dots, s_m\}$ be the recursive index functions of f . We say they terminate over the select condition r iff:

$$\forall (c_1, \dots, c_n) \in Z^n, \forall (j_1, j_2, \dots) \in \{1, \dots, m\}^N, \exists k > 0 :$$

$$r(s_{j_k} \circ \dots \circ s_{j_1}(c_1, \dots, c_n)) = \text{true}.$$

Definition 5.3 (Inductive variables). An index variable x_i is inductive in function $f(x_1, \dots, x_n)$ iff there exists a recursive function call $f(s_j(x_1, \dots, x_n))$ in the definition of f such that the i -th component of $s_j(x_1, \dots, x_n)$ is not syntactically equal to x_i .

Inductive functions break Halide’s formal rule against self-references in pure definitions[24]. They also violate the *syntactic separation restriction* described in the formalization of Halide[24]. This restriction requires self-references to $f(x_1, \dots, x_n)$ to set x_i as the i -th index of the self-referential call’s arguments if x_i appears at all in the corresponding update stage of $f(x_1, \dots, x_n)$.

The compiler statically checks that recursion reaches the base case from all inductive variable assignments. For example:

$$f(x) = \text{select}(x < 1, 1, f(x-1) + f(x+1))$$

is rejected because the recursion does not terminate.

To simplify the analysis and compilation process, we conservatively require that the inductive function’s indices are monotonically decreasing in recursive calls conditioned on the termination function r .

Definition 5.4 (Monotonicity). An index map $s_j : Z^n \rightarrow Z^n$ is monotonically decreasing conditioned on $r : Z^n \rightarrow \{\text{true}, \text{false}\}$ iff:

$$\forall i \in \{1, \dots, n\}, \quad (s_j(x))_i \leq x_i,$$

and $s_j(x_1, \dots, x_n) \neq (x_1, \dots, x_n)$ for all $(x_1, \dots, x_n) \in Z^n$ such that $r(x_1, \dots, x_n)$ evaluates to true.

For a function $f(x, y)$, example recursive calls that are allowed include $f(x-1, y)$ or $f(x, y-2)$ but not $f(x+1, y-1)$.

To perform this monotonicity check, we use Halide’s built-in `simplify` function to determine, for every pair of i and j , whether $(s_j(x))_i \leq x_i$ and $(s_j(x))_i < x_i$ simplify to true conditioned on $r(x_1, \dots, x_n)$ being true.

To ensure that the base case is reachable, we also require that the function satisfy the monotonic exit property.

Definition 5.5 (Monotonic exit property). If the following holds for all assignments of non-inductive variables, we say that an inductive function with termination condition r satisfies the *monotonic exit property*. Given an inductive function with indices $(x_1, \dots, x_n)$, arbitrarily fix the value of each non-inductive variable index. For each inductive variable x_i , there exists an expression b_i that does not vary with any inductive variable, such that $r(c_1, \dots, c_n)$ is true if at least one inductive variable assignment c_i satisfies $c_i < b_i$.

To check that a function satisfies this property, we use Halide’s `solve_for_outer_interval` function. Given a boolean condition c and variable x , `solve_for_outer_interval(c, x)` returns a range that is as tight as possible where c is guaranteed to be false if x is outside the range. We use `solve_for_outer_interval` on the condition $\overline{r(x_1, \dots, x_n)}$ (the complement of $r(x_1, \dots, x_n)$) and each variable x_i that is syntactically contained in the definition of $r(x_1, \dots, x_n)$. To satisfy the monotonic exit property, the range returned by `solve_for_outer_interval` for each x_i should have a finite lower bound that does not depend on any inductive variables in the definition of f .

As a simple example, the Fibonacci numbers can be defined as

$$f(x) = \text{select}(x < 1, x, f(x-1) + f(x-2)).$$

If we also define $g(x) = f(x)/2$; `f.store_root()`, this creates a two-stage pipeline with loop fusion. `f` is by default computed at each element of `g`. We need `f.store_root()` to stop previously computed values from being discarded. For comparison, we can remove loop fusion and compute `f` and `g` separately by specifying `f.compute_root()`.

5.1.2 Compilation

We lay out how Halide compiles inductive functions, including the sliding window and storage folding compiler optimizations. These optimizations are built into the Halide compiler, and no changes were made to them in the process of implementing inductive functions. The effects of these optimizations are visualized in Figure 5.2.

Loop nest construction. The start of the Halide compilation process, represented by the **Loop nests** stage in Figure 5.1, involves the construction of loop nests where functions are computed. In these loop nests, Halide uses bounds inference to determine boxes of function indices. (For a function with n arguments, a *box* is a n -dimensional interval of integers over which it is computed.) We make simple modifications to this process to compile an inductive function. First, Halide treats the inductive function as a pure function with no self-references to find an initial box to compute over. This process is described in the *Lowering* and *Bounds inference* sections of *Formal Semantics for the Halide Language* [24] and is not repeated here. Let the lower and upper bounds of the box be $(l_1, \dots, l_n)$ and $(u_1, \dots, u_n)$ respectively.

Halide expands the computed box to start computing at the base case of the recursion, replacing $(l_1, \dots, l_n)$ with some set $(b_1, \dots, b_n)$ such that $b_i \leq l_i$ for all i and $(u_1, \dots, u_n)$ satisfies the monotonic exit property with respect to $(b_1, \dots, b_n)$ (which is defined at the end of Section 5.1.1). This ensures that the dependencies of the recursion’s output are all computed. Halide iterates over boxes monotonically, so the function values are computed in a correct ordering. Since inductive functions are represented as pure functions during compilation, it is hypothetically possible for the compiler to make the computation order non-monotonic, which is safe for pure functions but not for inductive functions. From examining Halide’s compiler optimizations, we observe that Halide’s compilation process does not do this except when the inductive function is explicitly parallelized or vectorized over an inductive variable. This can only be done by the programmer before the compilation stage. Our implementation disallows this type of parallelization and vectorization for inductive functions, but allows inductive functions to be parallelized or vectorized over non-inductive variables.

After defining the initial loop nests, Halide performs several optimizations on the program’s intermediate representation. These steps are represented by the **Optimization** stages in Figure 5.1. Below, we describe the most relevant optimizations and how they interact with inductive functions.

Sliding-window optimization. The naive compilation above can cause values to be re-computed unnecessarily, performing asymptotically more work than necessary. To resolve this, the Halide compiler already has a mechanism to eliminate redundant re-computation: sliding-window optimization. Halide detects sliding windows, which are pairs of loops that compute functions over overlapping ranges. In sliding window optimization, Halide shrinks the box of computation so that each value is computed only once. Here is a simplified description of how the sliding window mechanism works, excluding several complex checks and edge cases: Consider a block B in the Halide IR that produces function f over a box with lower and upper bounds $(l_1, \dots, l_n)$ and $(u_1, \dots, u_n)$. f can be pure or inductive. Suppose there is a loop over some variable y that contains B in its loop body such that exactly one dimension of the box, which we represent as k , has bounds that depend on y ; represent these bounds as $l_k(y)$ and $u_k(y)$. If values of f are not discarded between iterations over y , $l_k(y)$ and $u_k(y)$ are non-decreasing functions in y , and $l_k(y) < u_k(y - 1)$, the sliding window optimization replaces $l_k(y)$, the lower bound of dimension k , with $u_k(y - 1)$, and adds extra iterations to compute $l_k(y_{init})$ through $u_k(y_{init} - 1)$, where y_{init} is the initial value of y over its loop.

Storage folding. Inductive functions can also be optimized through Halide’s storage folding optimization. By default in Halide, the entire sequence of recursively computed values needs to be completely stored in memory even if each value only needs to be read a single time. Storage folding reduces unnecessary memory usage: it shrinks the buffer so it has just enough space to store the values that are required in a given loop iteration.

The following is, again, a simplified description of Halide’s storage folding mechanism. Consider a loop over y that contains a block where function f is produced and consumed. For simplicity, we consider the loop over y in isolation, ignoring any outer loops outside y ; the mechanism has separate checks to determine which loops are safe to perform storage folding over. Halide computes a box that contains at least the set of indices in f that are read and written in each iteration of y ; let its bounds be $(l'_1(y), \dots, l'_n(y))$ and $(u'_1(y), \dots, u'_n(y))$. If there is a dimension k such that $l'_k(y)$ is monotonically non-decreasing in y , then values with index x at dimension k are never used again after iteration y' where $x < l'_k(y')$, so they can be discarded. The amount of space that must be allocated for f in dimension k is $s := \max_y u'_k(y) - l'_k(y)$. All values that index into dimension k of f are taken modulo s .

This folding can often be automatically applied by the Halide compiler, but Halide has a command called `fold_storage` that allows the programmer to perform storage folding.

5.1.3 Fibonacci example

We walk through the compiler optimizations visualized in Figure 5.2. Consider the same Fibonacci definition:

$$f(x) = \text{select}(x < 1, x, f(x-1) + f(x-2)).$$

After the initial compilation step, the Halide IR for computing this function and dividing the Fibonacci numbers by 2 can be represented by the following pseudocode. The inner loop’s bounds of $\min(1, i)$ to i can be explained as follows: If $i > 1$, then the bounds are from 1 to i , reflecting the fact that the base case occurs when $i < 2$. If i is 0 or 1, the schedule is in the base case, so only a single value is computed.

```

1 //Produce g:
2 for i = 0 to n
3     //Produce f:
4     for j = min(1, i) to i
5         f[j] = select(j < 2, j, f[j-1] + f[j-2])
6     g[i] = f[i] / 2

```

After the sliding window optimization, it can be represented as:

```

1 //Produce g:
2 for i = 0 to n
3     f[i] = select(i < 2, i, f[i-1] + f[i-2])
4     g[i] = f[i] / 2

```

Finally, after storage folding with a factor of 3, it can be represented as follows. It is theoretically possible to perform folding with a factor of 2, but Halide cannot do this for

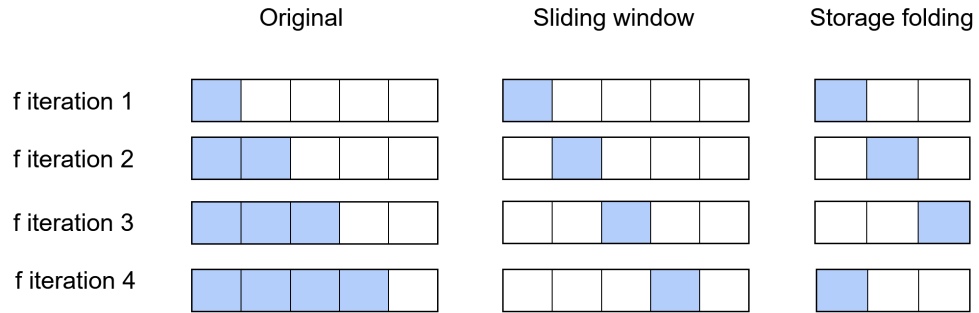


Figure 5.2: Comparison of computed values for an inductively defined Fibonacci function with and without various compiler optimizations. The filled-in boxes represent computed values during each loop iteration.

reasons discussed in Section [7.2.3](#)

```

1  for i = 0 to n
2      //Produce f:
3      f[i % 3] = select(i < 2, i,
4                      f[(i-1) % 3] + f[(i-2) % 3])
5      g[i] = f[i % 3] / 2

```

Chapter 6

Evaluation

We test FlashTile on three different tasks to showcase its abilities, demonstrating that it can create a variety of optimized algorithms, including algorithms with new optimizations. On top of FlashTile, we use additional Halide scheduling commands to further optimize the programs. We use FlashTile to replicate the tiling strategy of the original version of FlashAttention, replicate two schedules for the StereoBM algorithm, and improve the runtime of a self-attention layer for time series prediction.

In all of these tasks, FlashTile uses the same infrastructure to perform tiling and derive a merge function, but uses different infrastructure to combine the tiles. All experiments for FlashTile were run on a single core of an Intel Xeon E5-2695 v2 CPU.

For inductive functions, we have also run experiments that demonstrate they can represent schedules that are faster than optimized schedules in current Halide. We use inductive functions to optimize the StereoBM algorithm to get similar performance to OpenCV’s implementation, and we also use inductive functions to optimize a pipeline with two functions, one of which is recursively defined.

6.1 FlashTile for FlashAttention

The first experiment focuses on optimizing the attention algorithm with FlashTile. Given matrices Q , K , and V , attention can be defined as $\text{softmax}(QK^T)V$, where softmax is performed on each row of QK^T . This computation can be easily tiled over the rows of QK^T and thus the rows of Q , but not the columns of QK^T , since each element of $\text{softmax}(QK^T)$ depends on all the elements in the same row as QK^T .

Attention can be implemented in Halide as follows:

Listing 6.1: Attention in Halide

```

1 Var x, y;
2 Func S, rmax, rowsum, P1;
3 RDom r1(0, Q.dim(1).extent()), r(0, K.dim(0).extent());
4 S(y,x) = sum(Q(y,r1)*K(x, r1));
5 rmax(y) = (float)-INFINITY;
6 rmax(y) = max(rmax(y), S(y,r));
7 rowsum(y) = sum(exp(S(y,r)-rmax(y)));
8 P1(y,x) = exp(S(y,x)-rmax(y))/(rowsum(y));
9 out(y,x) = sum(P1(y, r)*V(r, x));

```

FlashAttention is an algorithm that tiles attention across both rows and columns, enabling increased locality and a reduction in high-bandwidth memory transfers on GPUs.

Without any scheduling commands, FlashAttention’s tiling strategy can be implemented in Halide as follows:

Listing 6.2: FlashAttention in Halide

```

1 Var x, x2, y, z;
2 Func S, rmax_1, rowsum_1, block_1, out_calc;
3 RDom r1(0, Q.dim(1).extent()), r2(0, blocksize);
4 RDom r3(1, (K.dim(0).extent() / blocksize) - 1);
5
6 S(y,x) = sum(Q(y,r1)*K(x, r1));
7 rmax_1(y, x2) = (float)-INFINITY;
8 rmax_1(y, x2) = max(rmax_1(y, x2), S(y, x2*blocksize+r2));
9 rowsum_1(y, x2) = sum(exp(S(y,x2*blocksize+r2)-rmax_1(y, x2)));
10 block_1(y, x2, z) = sum(exp(S(y,x2*blocksize+r2)-rmax_1(y, x2))*V(x2*
    blocksize+r2, z))/rowsum_1(y, x2);
11 out_calc(y, z) = Tuple(rmax_1(y, 0), rowsum_1(y, 0), block_1(y, 0, z));
12 Expr new_max = max(out_calc(y, z)[0], rmax_1(y, r3));
13 Expr new_rowsum = exp(out_calc(y, z)[0]-new_max)*out_calc(y,z)[1] + exp(
    rmax_1(y, r3)-new_max)*rowsum_1(y, r3);
14 Expr new_block = (exp(out_calc(y, z)[0]-new_max)*out_calc(y,z)[2]*
    out_calc(y,z)[1]+exp(rmax_1(y, r3)-new_max)*block_1(y, r3, z))/
    new_rowsum;
15 out_calc(y, z) = Tuple(new_max, new_rowsum, new_block);
16 out(y, z) = out_calc(y,z)[2];

```

The FlashAttention algorithm can be defined as follows, adapted from the original paper. For convenience, vector operations using l , m , and other vectors are performed pointwise.

Algorithm 9 FlashAttention algorithm

```
1: Let the dimensions of  $Q, K, V$  be  $N \times d$  matrices
2: Divide  $Q$  into  $k$  blocks  $Q_1, \dots, Q_k$ 
3: Divide  $K, V$  into  $L$  blocks  $K_1, \dots, K_L, V_1, \dots, V_L$ 
4: Initialize  $l$  as a  $N$ -element vector with all zeros
5: Initialize  $m$  as a  $N$ -element vector with  $-\infty$ 
6: Initialize  $O$  as a  $N$  by  $N$  matrix with all zeros
7: Divide  $O$  into  $k$  blocks  $O_1, \dots, O_k$ 
8: for  $i$  in range( $1, k$ ) do
9:   for  $j$  in range( $1, l$ ) do
10:     $S_{ij} \leftarrow Q_i K_j^T$ 
11:     $\tilde{m}_{ij} \leftarrow \text{rowmax}(S_{ij})$ 
12:     $\tilde{l}_{ij} \leftarrow \text{rowsum}(\exp(S_{ij} - \tilde{m}_{ij}))$ 
13:     $P_{ij} \leftarrow \exp(S_{ij} - \tilde{m}_{ij}) / \tilde{l}_{ij}$ 
14:     $R_{ij} \leftarrow P_{ij} V_j$ 
15:     $m'_i \leftarrow \max(m_i, \tilde{m}_{ij})$ 
16:     $l'_i \leftarrow \exp(m_i - m'_i) l_i + \exp(\tilde{m}_{ij} - m'_i) \tilde{l}_{ij}$ 
17:     $O_i \leftarrow (l_i \exp(m_i - m'_i) O_i + \tilde{l}_{ij} \exp(\tilde{m}_{ij} - m'_i) R_{ij}) / l'_i$ 
18:     $l_i \leftarrow l'_i$ 
19:     $m_i \leftarrow m'_i$ 
20:   end for
21: end for
22: Return  $O$ 
```

The inner loop performs the regular attention algorithm on a single tile of the intermediate matrix S and computes the auxiliary statistics $\tilde{m}$ and $\tilde{l}$. Next, it merges the results of this tile with the results of previous tiles, updating O_i , m_i , and l_i .

6.1.1 Construction

We use FlashTile to semi-automatically transform attention into FlashAttention. The implementation is shown in listing 6.3. The Halide program starts by defining attention using the simple and unoptimized definition. It then calls the `do_merge` and `reduction_transform` methods to transform the function into FlashAttention. Finally, the program calls scheduling commands that already exist in Halide so that FlashAttention is computed efficiently.

Listing 6.3: FlashTile implementation of FlashAttention definition

```
1 // Inputs: Q, K, V, size, tiledim
2 Var i, j, pure, ji, jo, joo, jooo;
3 RDom rmul(0, 64), rowvar(0, size), tile_r(0, tiledim), replace_r(0, size
  / tiledim);
4 K_chunked(j, i) = K(j, i);
5 V_chunked(i, j) = V(i, j);
6 S(j, i) = Halide::cast<float> > (Expr((float) 0.0));
```

```

7  S(j, i) += Q(j, rmul) * K_chunked(i, rmul);
8  rowmax(j) = Halide::cast<float>((float) -INFINITY);
9  rowmax(j) = max(rowmax(j), S(j, rowvar));
10 rowsum(j) = Halide::cast<float>(Expr((float) 0.0));
11 rowsum(j) = rowsum(j) + Halide::cast<float>(exp(S(j, rowvar) - rowmax(j))
    );
12 P1(j, i) = Halide::cast<float>(Expr(0.0));
13 P1(j, i) += exp(S(j, rowvar) - rowmax(j)) * V_chunked(i, rowvar) / rowsum
    (j);
14
15 MergeOut combiner = do_merge(P1, {rowvar.x});
16 TileFinalResult result = P1.reduction_transform(combiner, {P1, rowsum,
    rowmax}, false, v, {replace_r});
17
18 std::map < std::string, Func > func_map = result.func_name_map;
19 func_map[rowmax.name()].update(0).reorder(tile_r, j, pure);
20 func_map[rowsum.name()].update(0).reorder(tile_r, j, pure);
21 func_map[P1.name()].update(0).reorder(tile_r, i, j, pure);
22
23 RVar tile_ro;
24
25 result.raw_out.split(j, joo, j, 64).split(joo, jooo, joo, size / 64).
    reorder(i, j, joo, jooo);
26 result.raw_out.update(0).split(j, joo, j, 64).split(joo, jooo, joo, size
    / 64).reorder(i, j, joo, jooo, result.reductions[0]);
27 result.intm.in(result.raw_out).copy_to_host().compute_at(result.raw_out,
    joo).store_in(MemoryType::Heap);
28
29 func_map[rowmax.name()].compute_at(result.raw_out, joo);
30 func_map[rowsum.name()].compute_at(result.raw_out, joo);
31 func_map[P1.name()].compute_at(result.raw_out, joo).store_in(MemoryType::
    Heap);
32 func_map[rowmax.name()].update(0).gpu_tile(j, jo, ji, 32).reorder(ji, jo)
    ;
33 func_map[rowsum.name()].update(0).gpu_tile(j, jo, ji, 32).reorder(ji, jo)
    ;
34 func_map[P1.name()].gpu_tile(j, jo, ji, 32).reorder(i, ji, jo);
35 func_map[P1.name()].update(0).gpu_tile(j, jo, ji, 32).reorder(i, ji, jo);
36
37 func_map[rowmax.name()].in(result.intm).copy_to_host().compute_at(result.
    raw_out, joo);
38 func_map[rowsum.name()].in(result.intm).copy_to_host().compute_at(result.
    raw_out, joo);
39 func_map[P1.name()].in(result.intm).copy_to_host().compute_at(result.
    raw_out, joo);

```

```

40 S.compute_at(result.raw_out, joo).store_in(MemoryType::Heap);
41 S.gpu_tile(j, jo, ji, 32).reorder(i, ji, jo);
42 S.update(0).gpu_tile(j, jo, ji, 32).reorder(i, ji, jo);
43
44 Q_chunked.compute_at(result.raw_out, joo);
45 K_chunked.compute_at(result.raw_out, jooo);
46 V_chunked.compute_at(result.raw_out, jooo);
47 Q_chunked.in().copy_to_device().compute_at(result.raw_out, joo);
48 K_chunked.in().copy_to_device().compute_at(result.raw_out, jooo);
49 V_chunked.in().copy_to_device().compute_at(result.raw_out, jooo);
50
51 result.raw_out.parallel(joo);
52 result.raw_out.update(0).parallel(joo);
53 result.raw_out.compute_root();

```

Using the previous description of the implementation of FlashTile, every element of O is part of a tuple that also contains values of auxiliary functions m and l . However, the values of m and l depend only on the row of O , not the column. That is, only one value of m and l is needed per row of O , so it is unnecessary to store and use a separate copy for every element of O . Instead, FlashTile detects this redundancy and eliminates it by computing m and l in the first column of O and using the same values of m and l for all other columns of O . By removing this redundancy, the cost of the merge function is substantially reduced, mainly by reducing the number of expensive `exp` operations. Our improvements to loop-invariant code motion in the Halide compiler, described in Section 4.4, were an important factor in producing this performance boost.

6.1.2 Experimental results

To test attention and FlashAttention, we simulate high-bandwidth memory (HBM) transfers by forcing the programs to load and store to and from CPU memory. FlashTile dramatically reduces the amount of simulated HBM transfers on the GPU, demonstrating that it correctly produces FlashAttention. Figure 6.1 shows the total amount of data transferred to and from high-bandwidth memory for unoptimized attention (represented by dotted lines) and FlashAttention (represented by solid lines) for a range of parameter sizes. The *theory* points represent the predicted amount of transferred memory following the analysis in the FlashAttention paper[18]. However, due to the limitations explained in Section 4.5, our schedule is less efficient than the actual FlashAttention schedule due to processing each tile multiple times, so our theoretical analysis results in substantially higher total memory transfers. The non-theory points represent the measured amount of memory transferred in our simulations. We vary the sizes of the tiles and the input matrices. As tile size increases, the total data transferred when computing FlashAttention decreases, as predicted by theoretical calculations. The total data transferred when computing unoptimized attention barely changes at all with respect to the tile size, which is consistent with theoretical calculations.

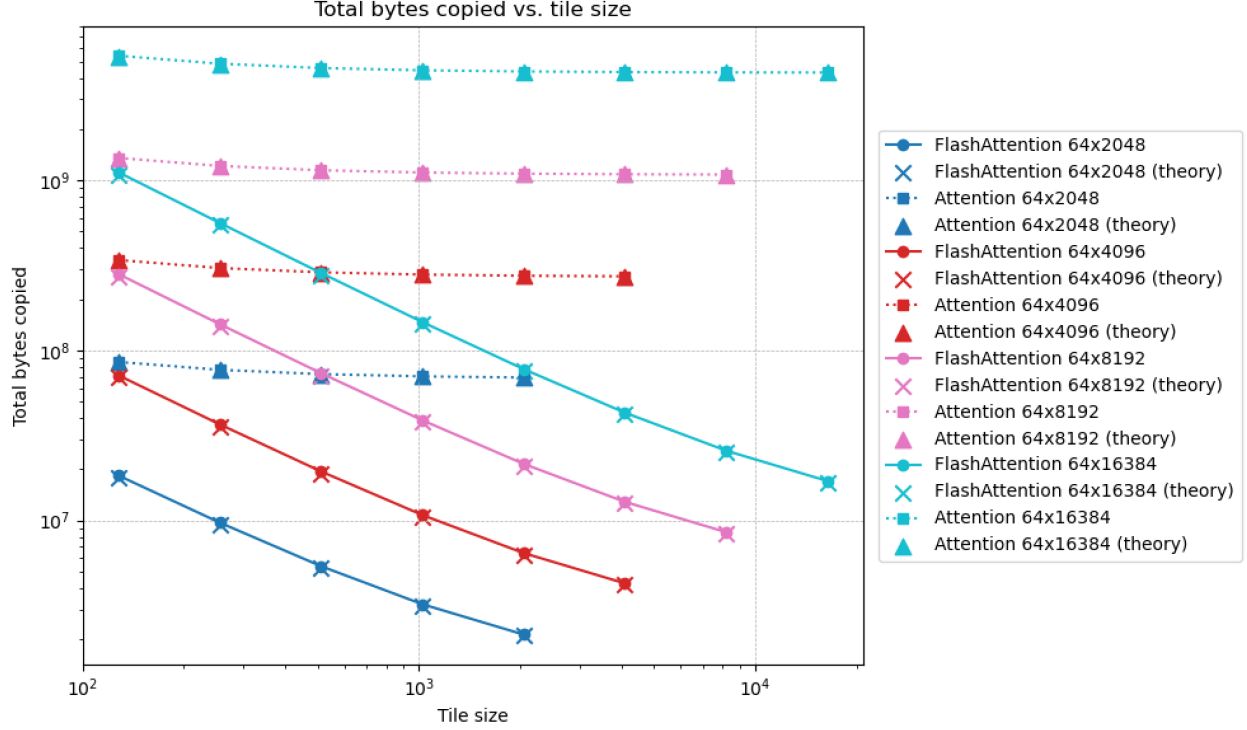


Figure 6.1: Memory transfers for unoptimized attention and FlashAttention.

6.2 FlashAttention in a time series

In an extension of the previous example, we use FlashTile to optimize an algorithm consisting of a combination of tiling types: sliding-window aggregations and complex reductions. Substantial research has been done on applying transformers and similar models for time series prediction. Informer is a variant of the transformer that uses an approximation algorithm to calculate attention more efficiently [25].

Many time series forecasting models make forecasts continuously, processing the input windows in temporal order. This continuous operation is sometimes used during model training in the domain of online learning [26]. This means that adjacent windows can have substantial overlap, resulting in redundant work.

InTrans [27] is a variant of Informer that uses an optimized sliding window algorithm to efficiently calculate the Q , K and V matrices that are input to FlashAttention. Using FlashTile, we extend InTrans and Informer to remove redundant work from the attention layer itself, which can be written in the form of a sliding window. In this experiment, self-attention is computed for a range of t on windows $D[t, w + t]$, where D is the dataset, w is the window size, and t is the timestep. This experiment uses the same assumption as InTrans that the model processes the input windows in sequential temporal order. Continuous processing is common during testing and evaluation, given the popularity of sliding-window aggregation algorithms [28].

6.2.1 Construction

Apart from using full self-attention instead of the Informer’s approximation of self-attention, this experiment’s schedule is based on the first layer of Informer. It aims to replicate the first layer of a typical transformer-based time-series model.

The Halide definition is as follows:

Listing 6.4: Halide definition of the first layer of Informer

```
1 // inputs: series, EmQ, EmK, EmV, LQ, LK, LV, winsize, slide=1
2 Var i, j, t, t2, h;
3 RDom rmul_input(0, series.height());
4 RDom rmul_linear(0, EmQ.width());
5 int sl = min(slide, 8);
6 series_ext(j,i)=BoundaryConditions::constant_exterior(series, 0.0f)(j,i);
7 preq(j, i)=sum(series_ext(j, rmul_input.x) * EmQ(i, rmul_input.x));
8 prek(j, i)=sum(series_ext(j, rmul_input.x) * EmK(i, rmul_input.x));
9 prev(j, i)=sum(series_ext(j, rmul_input.x) * EmV(i, rmul_input.x));
10 Q(j, i, h)=sum(preq(j, rmul_linear.x) * LQ(i,rmul_linear.x, h));
11 K(j, i, h)=sum(prek(j, rmul_linear.x) * LK(i,rmul_linear.x, h));
12 V(i, j, h)=sum(prev(j, rmul_linear.x) * LV(i,rmul_linear.x, h));
13
14 RDom rmul(0, dims, "rmul");
15 S(j, i, h) = Halide::cast<float>(Expr((float)0.0));
16 S(j, i, h) += Q(j, rmul, h) * K(i, rmul, h);
17
18 RDom rowvar(0, winsize);
19 rowmax(j, t, h) = Halide::cast<float>((float)-INFINITY);
20 rowmax(j, t, h) = max(rowmax(j, t, h), S(j, rowvar + t * slide, h));
21 rowsum(j, t, h) = Halide::cast<float>(Expr((float)0.0));
22 rowsum(j, t, h) = rowsum(j, t, h) + Halide::cast<float>(exp(S(j, rowvar +
    t * slide, h) - rowmax(j, t, h)));
23
24 P1(i, j, t, h) = Halide::cast<float>(Expr(0.0));
25 P1(i, j, t, h) += exp(S(j, rowvar + t * slide, h) - rowmax(j, t, h)) * V(
    i, rowvar + t * slide, h) / rowsum(j, t, h);
26 out(j, i, t, h) = P1(i, j + t * slide, t, h);
27
28 conv(j,i,t,h) = out(j - 1, i, t, h) + out(j, i, t, h) + out(j + 1, i, t,
    h);
29 activ(j, i, t, h) = max(0, conv(j, i, t, h));
30 pool(j, i, t2, h) = max(activ(2 * j + 1, i, t2, h), activ(2 * j + 2, i,
    t2, h));
```

In our second experiment, we use FlashTile to substantially improve the performance of self-attention over a sliding window. The program begins by defining a naive version of

attention (not FlashAttention) that is computed for a certain number of timesteps. Using FlashTile, the program calls a method, `bdbbs_transform`, that divides the computation of attention into tiles, converting it into FlashAttention. This method applies the bidirectional box-sum algorithm as defined in Section 3.3 to efficiently combine tiles with only a small amount of computation per window. Afterwards, the program adds predicates to the tiled version of self-attention so the program only computes the tiles that have not been computed at any previous timestep. This final step relies on compiler modifications as described in Section 4.4. One drawback of this approach is that it requires much more memory compared to regular FlashAttention, up to several hundred megabytes for the largest window size we tested.

6.2.2 Experimental results

We ran this experiment using self-attention windows of varying sizes from 120 to 720. They are similar to the sizes used to test Informer. This experiment uses the default parameters from the Informer experiments [25]: Each input data point is embedded into a 512-element vector, the embedded data points are passed through a linear layer so that each vector in Q , K , and V has size 64, the self-attention layer has 8 heads, and the output of self-attention is passed through a convolution layer and a maxpool layer.

Our experiment measured the time to run the program for 200 timesteps. We ran each test case five times and took the minimum runtime out of five runs.

As shown in Figure 6.2, we found that the sliding window attention with a slide of 1 gained an 8x speedup over PyTorch on the largest window size of 720, and gained a 5x speedup on sizes 480 and 600. Using FlashTile, the relative speedup of sliding window self-attention over regular self-attention becomes larger as the window size grows larger. This is consistent with theoretical analysis: With a constant slide, tiled self-attention using a sliding window should have a linear time complexity, whereas regular self-attention is quadratic time.

6.3 FlashTile in the StereoBM algorithm

Finally, we demonstrate FlashTile’s optimization abilities on a third algorithm with sliding-window aggregations. The StereoBM (stereo box matching) algorithm is an image processing algorithm that takes a pair of offset 2D images. Its main computational cost is computing a 2D box sum: summing the elements in fixed-size windows defined by corners (x, y) and $(x + k, y + k)$ for every pixel (x, y) .

The naive algorithm, which independently computes the sums at every window, is highly inefficient. However, this computation can be expressed as a two-dimensional sliding window, allowing us to exploit redundant work using the methods described in Section 3.3. These schedules involve the computation of one-dimensional sums that represent columns of each two-dimensional window. The state-of-the-art implementation for StereoBM cannot be scheduled in Halide[22], but several alternative implementations can be scheduled. These alternative implementations all require different algorithms in Halide.

This box filter is an example of a two-dimensional *included-sums problem* [20]: for every coordinate (x, y) in some input array, we must compute the sum of all elements in a fixed-size

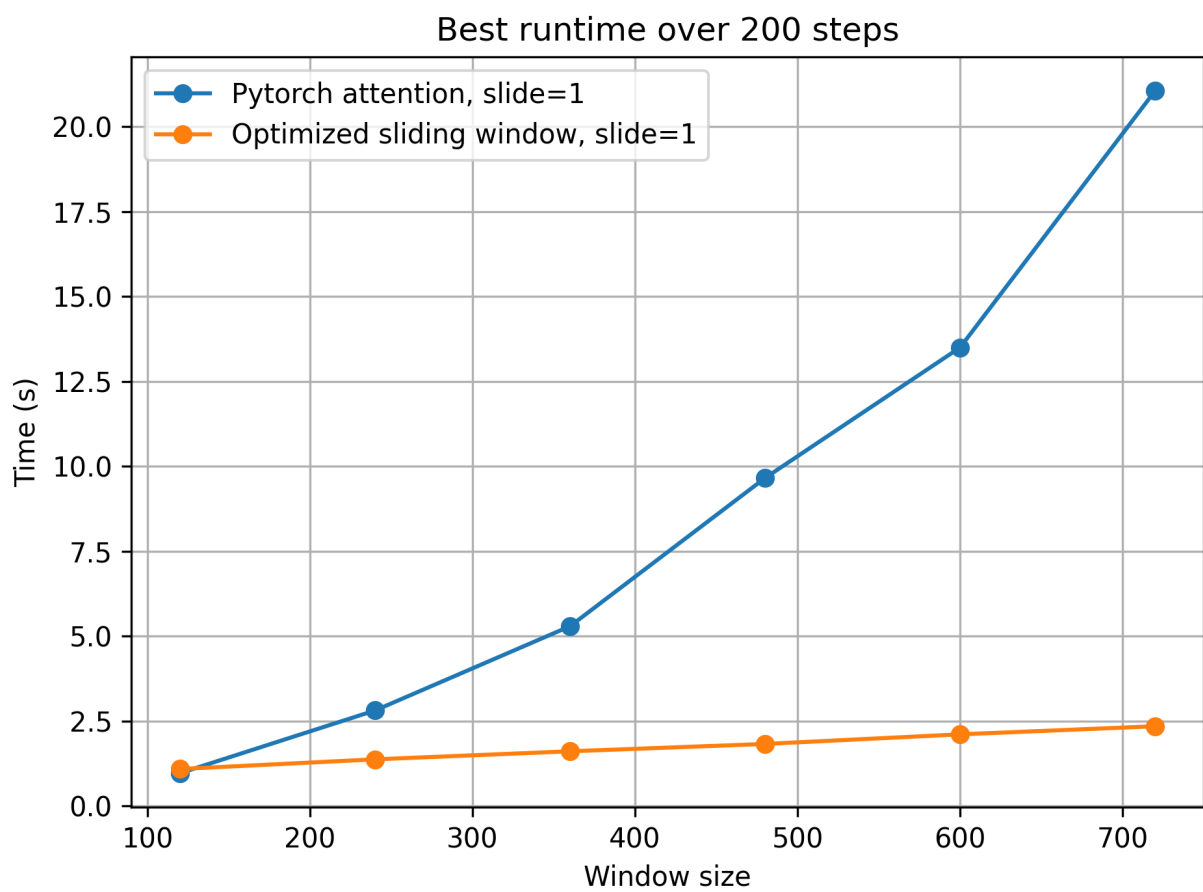


Figure 6.2: Runtimes of sliding attention in Halide and regular FlashAttention in PyTorch

window centered on (x, y) . Since the summation operator is addition, this operator has an inverse (subtraction), making this component of StereoBM a *weak* included-sums problem [20]. The schedules use the techniques described in Section 3.3.

We implemented schedules 2 and 3 in Zhang’s paper [22] for the StereoBM algorithm. Schedules 2 and 3 use different algorithms, which are both different from a naive implementation of a box filter. Schedule 2 constructs 1-dimensional sums in a direct fashion without using sliding windows, only using a sliding window to combine the 1-dimensional sums. This sliding window calculates the two-dimensional box sums. Since the 1-dimensional sums are calculated directly, the runtime of calculating these sums increases roughly linearly with the window size. Schedule 3 uses sliding windows to calculate both the one-dimensional sums and the two-dimensional box sums.

Schedules 2 and 3 apply a layer of tiling to the box filter’s output to improve locality, where each tile of the output is computed using a separate sliding-window aggregation. If the tile size is arbitrarily large, the runtime of calculating each sum in a sliding window does not depend on the window size: After computing the sum at the beginning of the tile, every other sum requires only two operations.

6.3.1 Construction

We use FlashTile to reconstruct two schedules for computing the box filter in the StereoBM algorithm. Without FlashTile, the Halide implementations of Schedules 2 and 3 require separate definitions for each schedule. Using FlashTile, we construct the two schedules from a single simple definition, which is the naive algorithm for computing the box filter. To replicate Schedule 2, the program calls `diff_transform` to divide the N-by-N box filter into 1-by-N tiles; this method also applies tiling to the box filter output as described at the end of the above subsection. `diff_transform` merges the 1-by-N tiles using difference-based aggregation.

To replicate Schedule 3, the program calls the same method to divide the N-by-N box filter into 1-by-N tiles, and calls this method a second time to divide the 1-by-N tiles into 1-by-1 tiles.

The paper does not describe the full programs for these schedules, so we made assumptions to fill in the gaps in our baseline implementations of these schedules. We use a tile size of 64 and an image depth of 32. Only the box filter step is replicated in this experiment, not the end-to-end StereoBM filter.

6.3.2 Experimental results

To ensure that FlashTile can replicate the schedules of the baseline implementations, we compared the runtimes across a range of window sizes. In all cases, the input was a 1024x1024 image composed of random data. Other parameters were also fixed. We ran each test case five times and took the minimum out of five runs.

Figure 6.3 shows that our implementations of the two schedules have runtimes similar to those of the baseline implementations, demonstrating that FlashTile can replicate schedules that are roughly equivalent to the baselines.

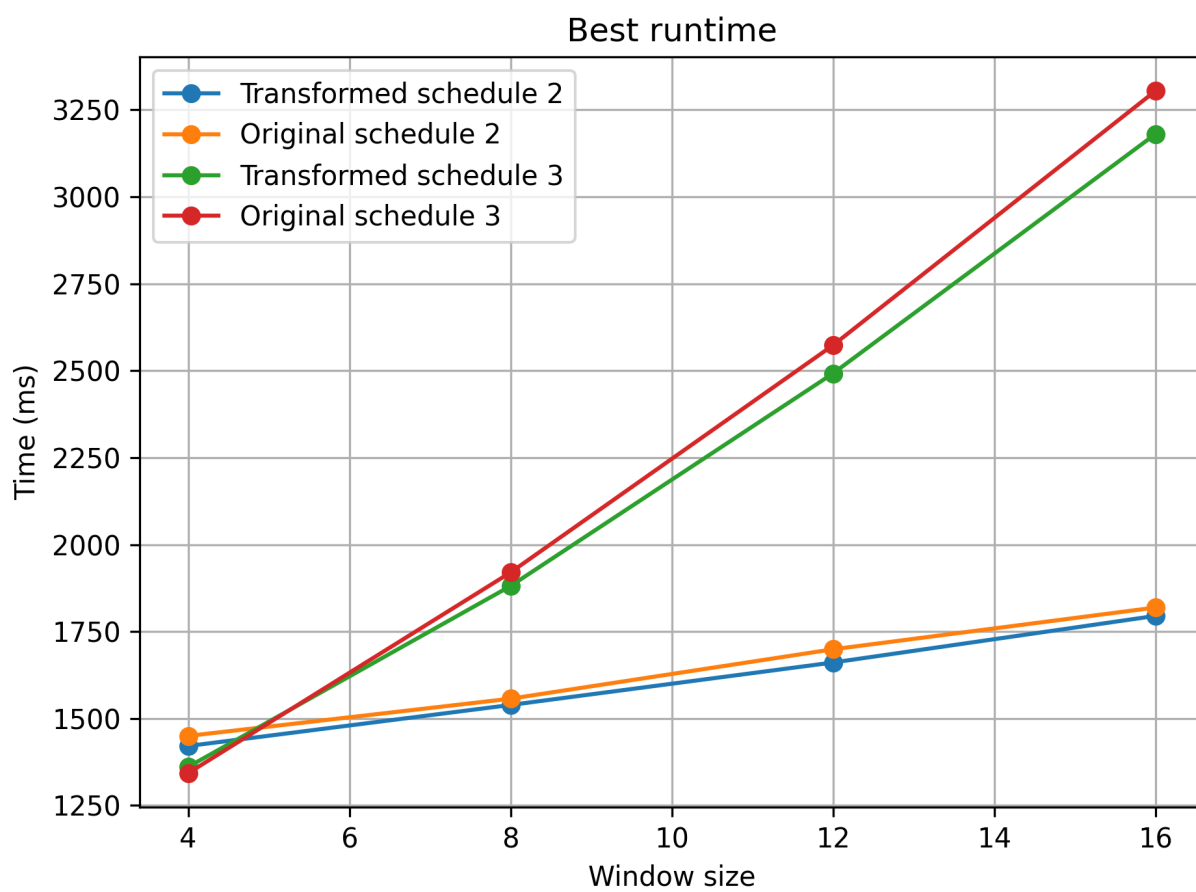


Figure 6.3: StereoBM schedule results

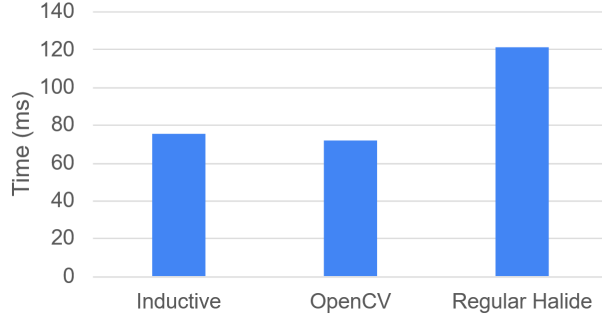


Figure 6.4: Runtime comparison for three implementations of the StereoBM algorithm

6.4 Benchmarks

In Table 6.1, we compare the lines of code in our transformed programs and their manually written equivalents. This list does not include sliding attention, the most complex program in our experiments and thus the one where we would expect the greatest benefit from FlashTile. However, this list shows that FlashTile can reduce the complexity of writing even relatively simple programs such as StereoBM.

Benchmark	Manual (lines)	FlashTile (lines)	Reduction (%)
FlashAttention	36	27	25%
StereoBM 2	15	14	7%
StereoBM 3	14	12	14%
Sliding attention	N/A	N/A	N/A

Table 6.1: Using FlashTile reduces the lines of code in the benchmarks. This comparison counts lines containing scheduling commands as well as lines used to define functions.

6.5 Inductive functions

In the first experiment for inductive functions, we compared three schedules for the StereoBM depth perception algorithm: a baseline implementation of StereoBM in Halide, an optimized implementation with inductive functions, and OpenCV’s implementation of StereoBM. This was a preliminary test that was run on a laptop (a Lenovo Yoga 9i) and not a dedicated server. The programs were run on a single core.

The optimized Halide schedule was designed to replicate OpenCV’s implementation, and the baseline schedule is an implementation of Schedule 3 from *Experimental Implementations of Stereo Matching Algorithms in Halide*. The execution time using inductive functions decreased by over 30% compared to a baseline Halide implementation, allowing Halide to nearly match the performance of OpenCV. The results are shown in Figure 6.4.

In the second experiment, we compared two Halide schedules for a simple two-stage pipeline, where the first stage computes a prefix sum of an input array and the second stage divides all elements of the prefix sum by 4. This is the example that is used in Figure 2.4. The baseline schedule and inductive schedule are shown in listing 6.5 and listing 6.6 respectively. The input length was 2^{26} 32-bit integers, which is enough for the array to not fit in cache. This experiment was run single-threaded on an Intel Xeon E5-2695 v2 CPU.

The results show that the inductive schedule is twice as fast as the non-inductive one. Without inductive functions, Halide cannot do loop fusion on this program. The resulting schedule has to compute the entire prefix sum before dividing by 4. With inductive functions, it only needs to compute one prefix sum element at a time, which is much faster if the array does not fit in cache. The non-inductive version has to store the intermediate array in main memory, which is slow.

Listing 6.5: Non-inductive pipeline schedule

```

1 // inputs: in, length
2 Var x, xo, xi;
3 RDom r(1, length - 1);
4 prefix(x) = undef<int>();
5 prefix(0) = in(0);
6 prefix(r) = in(r) + prefix(r-1);
7 out(x) = prefix(x) / 4;
8 out.bound(x, 0, length).split(x, xo, xi, 8, TailStrategy::RoundUp).
   vectorize(xi);
9 out.compute_root();
10 return out;
```

Listing 6.6: Inductive pipeline schedule

```

1 // inputs: in, length
2 Var x;
3 prefix(x) = select(x < 1, in(x), in(x) + prefix(x - 1));
4 out(x) = prefix(x) / 4;
5 out.bound(x, 0, length);
6 prefix.store_root().compute_at(out, x).fold_storage(x, 2);
7 return out;
```

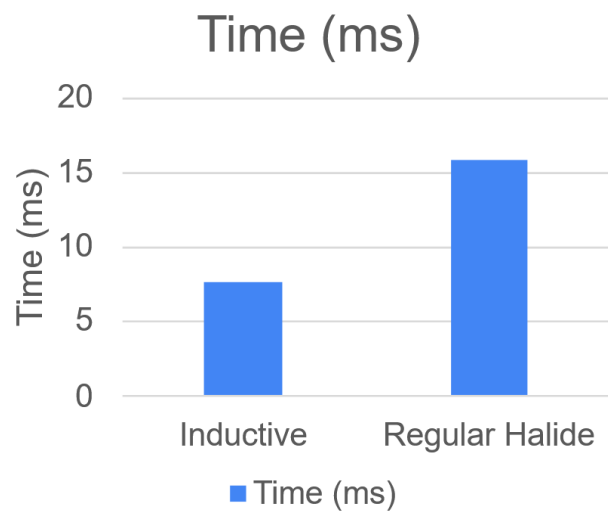


Figure 6.5: Runtime comparison for two implementations of the two-stage pipeline in Figure 2.4.

Chapter 7

Future Work

7.1 Future Work on FlashTile

One area that we do not address is the ability to FlashTile on other systems. The benefit of implementing FlashTile in Halide is that it integrates well with Halide’s other scheduling commands. However, optimizing algorithms in Halide is heavily constrained by the Halide language itself. For example, in Section 6.3, we described how the most efficient StereoBM schedule could not be implemented in Halide. To implement sliding-window attention in Halide, we needed to modify the Halide compiler to support certain predicate operations. A framework such as PyTorch would have more functionality than Halide, allowing us to apply FlashTile to perform a larger variety of optimizations.

Another problem is that the implementation of FlashAttention in Halide cannot achieve optimal performance because Halide does not support tensor core operations. It is feasible to integrate tensor core support into Halide, although complex pattern matching would be required to implement tensor core support as a scheduling command.

7.2 Future work on inductive functions

7.2.1 Parallelization and vectorization

There are many inductive functions that can be parallelized or vectorized over the inductive variable. For example, $f(x) = \text{select}(x < 4, 0, \text{in}(x) + f(x-4))$ can be parallelized in x with a factor of 4: one thread for each value modulo 4. Each value of f only depends on other values of f that have the same index modulo 4. As another example,

$$f(x) = \text{select}(x < 4, 0, \text{in}(x) + f(x-4) + f(x-5))$$

can be vectorized with a width of 4 elements, but not parallelized.

7.2.2 General recursion

Inductive functions currently only support recursive calls with monotonically decreasing indices, but there are many recursive functions with non-monotonically decreasing chains of

dependencies. One example is

$$f(x,y) = \text{select}(x < 0 \mid y < 0, 0, \text{input}(x,y) + f(x-1,y) + f(x+1,y-1))$$

. This function can be computed, since all chains of dependencies reach the base case, but the x variable does not monotonically decrease.

7.2.3 Improved storage folding

Halide's current implementation of storage folding is suboptimal for many recursive functions. In the Fibonacci example in Section 5.1.1, only two elements need to be stored at a time: when a new Fibonacci number $f(x)$ is computed, $f(x-2)$ will never be used again, so $f(x)$ can overwrite $f(x-2)$, leaving $f(x-1)$ as the other stored value. However, Halide's storage folding does not recognize when elements that are used in a computation can be discarded at the end of the computation, so Halide needs to store at least three elements instead of two.

Appendix A

Additional material

A.1 Proofs

Proposition A.1. If m satisfies the *merge property*, specifically

$$m(f(A), f(B)) = f(A, B)$$

for all disjoint A, B in the image of f , then

$$m(f(A), f(B)) = f(\text{inv}(f(A)), \text{inv}(f(B))),$$

where inv is an arbitrary function that satisfies the weak inverse property.

Proof. Assume that m exists. Since $\text{inv}(f(A))$ is injective with respect to $f(A)$, there is some function m' that satisfies

$$m(f(A), f(B)) = m'(\text{inv}(f(A)), \text{inv}(f(B)))$$

for all disjoint inputs A and B , so there is some function f' that satisfies

$$m(f(A), f(B)) = f'(\text{inv}(f(A)), \text{inv}(f(B))).$$

Thus, for all disjoint $A' = \text{inv}(f(A))$, $B' = \text{inv}(f(B))$ in the image of inv ,

$$\begin{aligned} f'(A', B') &= f'(\text{inv}(f(A)), \text{inv}(f(B))) \\ &= m(f(A), f(B)) \\ &= m(f(\text{inv}(f(A))), f(\text{inv}(f(B)))) \\ &= m(f(A'), f(B')) \\ &= f(A', B'), \end{aligned}$$

where the last equality follows from the merge property. This means that $f' = f$ over the range of possible inputs A', B' . Therefore, if a merge function m exists, it is necessarily equal to $f(\text{inv}(f(A)), \text{inv}(f(B)))$ for any valid inverse function. $\square$

Proposition A.2. If merge function

$$m(B_1, \dots, B_k) = f(\text{inv}(B_1), \dots, \text{inv}(B_k))$$

exists with respect to f , difference function $d(X, Y)$ exists, and f has an inverse g in its second element that satisfies $g(f(x, y), y) = x$, then $d(X, Y)$ can be written in the form $f^-(\text{inv}(X), \text{inv}(Y))$ where

$$f^-(a, b) = f(g(f(a), b)).$$

Proof. Note that, for a valid merge function m satisfying the merge property:

$$m(d(f(A), f(B)), f(B)) = f(A).$$

Since

$$\begin{aligned} m(d(f(A), f(B)), f(B)) &= f(\text{inv}(d(f(A), f(B))), \text{inv}(f(B))) \\ &= f(A), \end{aligned}$$

assume f has an inverse, g , in its second argument, so

$$\begin{aligned} g(f(A), \text{inv}(f(B))) &= g(f(\text{inv}(d(f(A), f(B))), \\ &\quad \text{inv}(f(B))), \text{inv}(f(B))) \\ &= \text{inv}(d(f(A), f(B))). \end{aligned}$$

Therefore,

$$\begin{aligned} d(f(A), f(B)) &= f(\text{inv}(d(f(A), f(B)))) \\ &= f(g(f(A), \text{inv}(f(B)))). \end{aligned}$$

and so for all x, y in the image of f ,

$$d(x, y) = f(g(f(\text{inv}(x)), \text{inv}(y))).$$

Thus, $f^-(a, b)$ can be expressed as

$$f^-(a, b) = f(g(f(a), b)),$$

again assuming that f has an inverse g in its second argument. □

A.2 Preparation of Call Tree Details

The first step of FlashTile is to modify the definition of the input function so that it is split into tiles; this step does not involve merging tiles. FlashTile's implementation of this step outputs a structure that maps the input function $\mathbf{f}$ to a tiled function $\mathbf{ft}$. If f directly or indirectly calls one or more intermediate functions $\mathbf{g1}, \dots$ that should also be tiled, this step also constructs tiled forms of these functions: $\mathbf{gt1}, \dots$. Function $\mathbf{ft}$ corresponds to the partitioning of the input array defined in Section 3.2. Each tile outputs a tuple of values

containing `ft` and possibly one or more *auxiliary functions* from `gt1, ...`. The first stage of FlashTile is constructed using Algorithm 10. This algorithm defines a method `tiler` that transforms the input function `f` into a tiled form and recursively transforms any intermediate functions contained inside `f`.

Algorithm 10 First stage of FlashTile

```

1: Inputs: function f, reduction variable r, pure variable outer, reduction variable inner
2: Initialize ft
3: vals  $\leftarrow$  f.values()
4: if f.uses_rdom(r) then
5:   vals  $\leftarrow$  vals.substitute_func(f, ft).append_args_in_func(ft, outer)
6:   vals  $\leftarrow$  vals.subst_args_in_func(ft, r, inner)
7: end if
8: for dep_func in get_dependencies(f) do
9:   if dep_func.uses_rdom(r) then
10:    // replace every instance of the dependency with its tiled version and append
    outer to its arguments
11:    vals  $\leftarrow$  vals.substitute_func(dep_func, tiler(dep_func))
12:    vals  $\leftarrow$  vals.append_args_in_func (tiler(dep_func), outer)
13:    vals  $\leftarrow$  vals.subst_args_in_func (tiler(dep_func), r, inner)
14:  end if
15: end for
16: vals  $\leftarrow$  vals.substitute(r, inner + outer * inner.extent)
17: args  $\leftarrow$  f.args().substitute(r, inner).append(outer)
18: ft  $\leftarrow$  make_func(args, vals)

```

References

- [1] P. Suriana, A. Adams, and S. Kamil. “Parallel associative reductions in halide”. In: CGO ’17. Austin, USA: IEEE Press, 2017, pp. 281–291. ISBN: 9781509049318.
- [2] R. Ji, Y. Zhao, Y. Xiong, D. Wang, L. Zhang, and Z. Hu. “Decomposition-based Synthesis for Applying Divide-and-Conquer-like Algorithmic Paradigms”. In: *ACM Trans. Program. Lang. Syst.* 46.2 (June 2024). ISSN: 0164-0925. DOI: [10.1145/3648440](https://doi.org/10.1145/3648440). URL: <https://doi.org/10.1145/3648440>.
- [3] Y. Tang, R. A. Chowdhury, B. C. Kuszmaul, C.-K. Luk, and C. E. Leiserson. “The pochoir stencil compiler”. In: *Proceedings of the Twenty-Third Annual ACM Symposium on Parallelism in Algorithms and Architectures*. SPAA ’11. San Jose, California, USA: Association for Computing Machinery, 2011, pp. 117–128. ISBN: 9781450307437. DOI: [10.1145/1989493.1989508](https://doi.org/10.1145/1989493.1989508). URL: <https://doi.org/10.1145/1989493.1989508>.
- [4] G. Chaurasia, J. Ragan-Kelley, S. Paris, G. Drettakis, and F. Durand. “Compiling high performance recursive filters”. In: *Proceedings of the 7th Conference on High-Performance Graphics*. HPG ’15. Los Angeles, California: Association for Computing Machinery, 2015, pp. 85–94. ISBN: 9781450337076. DOI: [10.1145/2790060.2790063](https://doi.org/10.1145/2790060.2790063). URL: <https://doi.org/10.1145/2790060.2790063>.
- [5] J. Ragan-Kelley, C. Barnes, A. Adams, S. Paris, F. Durand, and S. Amarasinghe. “Halide: a language and compiler for optimizing parallelism, locality, and recomputation in image processing pipelines”. In: *SIGPLAN Not.* 48.6 (June 2013), pp. 519–530. ISSN: 0362-1340. DOI: [10.1145/2499370.2462176](https://doi.org/10.1145/2499370.2462176). URL: <https://doi.org/10.1145/2499370.2462176>.
- [6] M. Steuwer, T. Koehler, B. Köpcke, and F. Pizzuti. *RISE & Shine: Language-Oriented Compiler Design*. 2022. arXiv: [2201.03611](https://arxiv.org/abs/2201.03611) [cs.PL]. URL: <https://arxiv.org/abs/2201.03611>.
- [7] Y. Ikarashi, G. L. Bernstein, A. Reinking, H. Genc, and J. Ragan-Kelley. “Exocompilation for productive programming of hardware accelerators”. In: *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. PLDI 2022. San Diego, CA, USA: Association for Computing Machinery, 2022, pp. 703–718. ISBN: 9781450392655. DOI: [10.1145/3519939.3523446](https://doi.org/10.1145/3519939.3523446). URL: <https://doi.org/10.1145/3519939.3523446>.
- [8] M. Bansal, D. Sharlet, J. Ragan-Kelley, and S. Amarasinghe. “Lightweight and Locality-Aware Composition of Black-Box Subroutines”. In: *Proc. ACM Program. Lang.* 9.PLDI (June 2025). DOI: [10.1145/3729292](https://doi.org/10.1145/3729292). URL: <https://doi.org/10.1145/3729292>.

- [9] F. Kjolstad, S. Kamil, S. Chou, D. Lugato, and S. Amarasinghe. “The tensor algebra compiler”. In: *Proc. ACM Program. Lang.* 1.OOPSLA (Oct. 2017). DOI: [10.1145/3133901](https://doi.org/10.1145/3133901). URL: <https://doi.org/10.1145/3133901>.
- [10] A. Liu, G. Bernstein, A. Chlipala, and J. Ragan-Kelley. “A Verified Compiler for a Functional Tensor Language”. In: *Proc. ACM Program. Lang.* 8.PLDI (June 2024). DOI: [10.1145/3656390](https://doi.org/10.1145/3656390). URL: <https://doi.org/10.1145/3656390>.
- [11] R. Yadav, A. Aiken, and F. Kjolstad. “DISTAL: the distributed tensor algebra compiler”. In: *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. PLDI 2022. San Diego, CA, USA: Association for Computing Machinery, 2022, pp. 286–300. ISBN: 9781450392655. DOI: [10.1145/3519939.3523437](https://doi.org/10.1145/3519939.3523437). URL: <https://doi.org/10.1145/3519939.3523437>.
- [12] R. Baghdadi, J. Ray, M. B. Romdhane, E. Del Sozzo, A. Akkas, Y. Zhang, P. Suriana, S. Kamil, and S. Amarasinghe. “Tiramisu: a polyhedral compiler for expressing fast and portable code”. In: *Proceedings of the 2019 IEEE/ACM International Symposium on Code Generation and Optimization*. CGO 2019. Washington, DC, USA: IEEE Press, 2019, pp. 193–205. ISBN: 9781728114361.
- [13] A. Rasch. “(De/Re)-Composition of Data-Parallel Computations via Multi-Dimensional Homomorphisms”. In: *ACM Trans. Program. Lang. Syst.* 46.3 (Oct. 2024). ISSN: 0164-0925. DOI: [10.1145/3665643](https://doi.org/10.1145/3665643). URL: <https://doi.org/10.1145/3665643>.
- [14] T. Chen et al. “TVM: an automated end-to-end optimizing compiler for deep learning”. In: OSDI’18. Carlsbad, CA, USA: USENIX Association, 2018, pp. 579–594. ISBN: 9781931971478.
- [15] M. Hall, C. Oancea, A. C. Elster, A. Rasch, S. Joshi, A. M. Tavakkoli, and R. Schulze. “Scheduling languages: A past, present, and future taxonomy”. In: *arXiv preprint arXiv:2410.19927* (2024).
- [16] K. Morita, A. Morihata, K. Matsuzaki, Z. Hu, and M. Takeichi. “Automatic inversion generates divide-and-conquer parallel programs”. In: PLDI ’07. San Diego, California, USA: Association for Computing Machinery, 2007, pp. 146–155. ISBN: 9781595936332. DOI: [10.1145/1250734.1250752](https://doi.org/10.1145/1250734.1250752). URL: <https://doi.org/10.1145/1250734.1250752>.
- [17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. “Attention is all you need”. In: *Proceedings of the 31st International Conference on Neural Information Processing Systems*. NIPS’17. Long Beach, California, USA: Curran Associates Inc., 2017, pp. 6000–6010. ISBN: 9781510860964.
- [18] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, and C. Ré. *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. 2022. arXiv: [2205.14135](https://arxiv.org/abs/2205.14135) [cs.LG]. URL: <https://arxiv.org/abs/2205.14135>.
- [19] J. Li, D. Maier, K. Tufte, V. Papadimos, and P. A. Tucker. “No pane, no gain: efficient evaluation of sliding-window aggregates over data streams”. In: *SIGMOD Rec.* 34.1 (Mar. 2005), pp. 39–44. ISSN: 0163-5808. DOI: [10.1145/1058150.1058158](https://doi.org/10.1145/1058150.1058158). URL: <https://doi.org/10.1145/1058150.1058158>.

- [20] H. Xu, S. Fraser, and C. E. Leiserson. “Multidimensional Included and Excluded Sums”. In: *Proceedings of the 2021 SIAM Conference on Applied and Computational Discrete Algorithms (ACDA21)*, pp. 182–192. DOI: [10.1137/1.9781611976830.17](https://doi.org/10.1137/1.9781611976830.17). eprint: <https://epubs.siam.org/doi/pdf/10.1137/1.9781611976830.17>. URL: <https://epubs.siam.org/doi/abs/10.1137/1.9781611976830.17>.
- [21] D. Bradley and G. Roth. “Adaptive Thresholding using the Integral Image”. In: *Journal of Graphics Tools* 12.2 (2007), pp. 13–21. DOI: [10.1080/2151237X.2007.10129236](https://doi.org/10.1080/2151237X.2007.10129236). eprint: <https://doi.org/10.1080/2151237X.2007.10129236>. URL: <https://doi.org/10.1080/2151237X.2007.10129236>.
- [22] M. Zhang. “Experimental Implementations of Stereo Matching Algorithms in Halide”. Master of Engineering thesis. Cambridge, MA: Massachusetts Institute of Technology, June 2016. URL: <https://commit.csail.mit.edu/papers/2016/min-zhang-meng-thesis.pdf>.
- [23] N. Nayak, X. Wu, T. O. Odemuyiwa, M. Pellauer, J. S. Emer, and C. W. Fletcher. “FuseMax: Leveraging Extended Einsums to Optimize Attention Accelerator Design”. In: *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. Los Alamitos, CA, USA: IEEE Computer Society, Nov. 2024, pp. 1458–1473. DOI: [10.1109/MICRO61859.2024.00107](https://doi.ieeecomputersociety.org/10.1109/MICRO61859.2024.00107). URL: <https://doi.ieeecomputersociety.org/10.1109/MICRO61859.2024.00107>.
- [24] A. Reinking, G. L. Bernstein, and J. Ragan-Kelley. “Formal semantics for the halide language”. In: *arXiv preprint arXiv:2210.15740* (2022).
- [25] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang. *Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting*. 2021. arXiv: [2012.07436](https://arxiv.org/abs/2012.07436) [cs.LG]. URL: <https://arxiv.org/abs/2012.07436>.
- [26] K. Lipianina-Honcharenko, Y. Bodyanskiy, N. Kustra, and A. Ivasechko. “OLTW-TEC: online learning with sliding windows for text classifier ensembles”. In: *Frontiers in Artificial Intelligence* Volume 7 - 2024 (2024). ISSN: 2624-8212. DOI: [10.3389/frai.2024.1401126](https://doi.org/10.3389/frai.2024.1401126). URL: <https://www.frontiersin.org/journals/artificial-intelligence/articles/10.3389/frai.2024.1401126>.
- [27] S. Bou, T. Amagasa, and H. Kitagawa. “InTrans: Fast Incremental Transformer for Time Series Data Prediction”. In: *Database and Expert Systems Applications*. Ed. by C. Strauss, A. Cuzzocrea, G. Kotsis, A. M. Tjoa, and I. Khalil. Cham: Springer International Publishing, 2022, pp. 47–61. ISBN: 978-3-031-12426-6.
- [28] S. Bou, H. Kitagawa, and T. Amagasa. “L-BiX: incremental sliding-window aggregation over data streams using linear bidirectional aggregating indexes”. In: *Knowl. Inf. Syst.* 62.8 (Aug. 2020), pp. 3107–3131. ISSN: 0219-1377. DOI: [10.1007/s10115-020-01444-5](https://doi.org/10.1007/s10115-020-01444-5). URL: <https://doi.org/10.1007/s10115-020-01444-5>.